

# 3DMeshNet: A three-dimensional differential neural network for structured mesh generation

Jiaming Peng<sup>1</sup>, Xinhai Chen<sup>2\*</sup>, Jie Liu

<sup>1</sup>Laboratory of Digitizing Software for Frontier Equipment, National University of Defense Technology, Changsha 410073, China

<sup>2</sup>National Key Laboratory of Parallel and Distributed Computing, National University of Defense Technology, Changsha 410073, China

## ARTICLE INFO

### Keywords:

Mesh generation  
Three-dimensional structured mesh  
Differential equations  
Neural network

## ABSTRACT

Mesh generation is a crucial step in numerical simulations, significantly impacting simulation accuracy and efficiency. However, generating meshes remains time-consuming and requires expensive computational resources. In this paper, we propose a novel method, 3DMeshNet, for three-dimensional structured mesh generation. The method embeds the meshing-related differential equations into the loss function of neural networks, formulating the meshing task as an unsupervised optimization problem. It takes geometric points as input to learn the potential mapping between parametric and computational spaces. After suitable offline training, 3DMeshNet can efficiently output a three-dimensional structured mesh with a user-defined number of quadrilateral/hexahedral cells through the feed-forward neural prediction. To enhance training stability and accelerate convergence, we integrate loss function reweighting through weight adjustments and gradient projection alongside applying finite difference methods to streamline derivative computations in the loss. Experiments on different cases show that 3DMeshNet is robust and fast. It outperforms neural network-based methods and yields superior meshes compared to traditional mesh partitioning methods. 3DMeshNet significantly reduces training times by up to 85% compared to other neural network-based approaches and lowers meshing overhead by 4 to 8 times relative to traditional meshing methods.

## 1. Introduction

Numerical simulation has become increasingly important in various fields of natural science research, promoting the development of modern applied scientific research and engineering technology [1,2]. These simulations typically require the use of meshes as a fundamental geometric framework to facilitate the analysis of physical phenomena via finite element methods or other numerical techniques. Mesh generation, defined as dividing a continuous computational space into meshes or elements for further numerical solutions, is a critical step in engineering simulations [3]. The quality of the generated mesh has a significant impact on the accuracy of numerical simulations [4]. Given the significant impact of mesh quality on these aspects, the rapid generation of high-quality meshes has become a primary concern.

Structured meshes are widely used in numerous simulations requiring precise element alignment for high-efficiency and high-precision [5]. They present multiple benefits: (1) Structured meshes do not require additional storage to connect mesh elements, resulting in higher efficiency in data access. (2) Their structured topological nature facilitates easy control over mesh cell sizes. (3) The regularity of their

structure facilitates the implementation of convolutional and pooling operations in neural networks [6]. Motivated by these benefits, extensive research has been done on structured mesh generation [7,8]. Algebraic methods calculate mesh interior points through algebraic interpolation, employing techniques like polynomial and Transfinite Interpolation (TFI) [9]. This approach is straightforward and enables rapid mesh generation. However, it can lead to mesh degradation in areas with complex geometries, resulting in issues such as intersections. To mitigate these problems, Partial Differential Equation (PDE) methods are commonly used to generate meshes for arbitrary boundaries [8]. Given the boundary values, PDE methods pose mesh generation as a boundary value problem governed by differential equations. Meshes created using PDE methods are of higher quality but require many computational resources and are time-consuming [10]. Thus, there is a significant demand for further research and development of a rapid and robust structured mesh generation method that balances speed and mesh quality.

In recent years, deep neural networks (DNNs) have achieved remarkable progress in fields such as flow-field prediction [11], mesh quality determination [3], and other physical problems [12]. A few

\* Corresponding author.

E-mail address: [chenxinhai16@nudt.edu.cn](mailto:chenxinhai16@nudt.edu.cn) (X. Chen).

<https://doi.org/10.1016/j.gmod.2025.101257>

Received 19 April 2024; Received in revised form 22 November 2024; Accepted 8 March 2025

Available online 25 March 2025

1524-0703/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

methods have been developed to incorporate neural networks into mesh generation tasks. Jilani et al. [13] proposed a two-dimensional (2D) initial mesh generated based on a Self-organizing map neural network for finite element analysis and calculation of mesh self-adaptive parameters, followed by adaptive tuning of the initial mesh using the Self-organizing map neural network. Liu et al. [14] introduced the ISplitter method, combining neural networks with segmentation lines to transform B-rep geometrical shapes into unstructured meshes and facilitating neural network training through recursive data generation. Soman S [15] employed Conditional Generative Adversarial Networks to ascertain the coordinates of mesh nodes for generating 2D and three-dimensional (3D) unstructured meshes. However, these methods often rely on large training datasets, which can be costly and complex to obtain, particularly for geometries of intricate shapes. Methods like MGNet [8], which utilize unsupervised learning for mesh generation, address the challenge of extensive manual training data requirements but still encounter problems such as unstable convergence and long training times. Additionally, pioneer researchers primarily focus on generating structured meshes in 2D, and developing neural network-based methods for 3D meshes remains an open problem.

In this paper, we introduce 3DMeshNet, a novel method for 3D structured mesh generation. 3DMeshNet employs a neural network embedded with 3D elliptic PDEs to learn the meshing rules for 3D mesh partitioning. Specifically, a well-designed neural network is developed to approximate the potential mapping between parametric and computational spaces. The 3D elliptic PDEs and surface fitting items are embedded in the loss function, acting as penalty terms to guide the optimization of network parameters. After suitable training, 3DMeshNet can efficiently generate meshes of any specified size for the corresponding geometry using forward propagation. To reduce training time, improve surface fitting accuracy, and enhance model convergence stability, we employed finite differences (FD), loss function reweighting, and gradient projection to improve the model from three aspects. Finite differences reduce computational costs and accelerate training by efficiently computing derivatives. Loss function reweighting improved surface representation by adjusting loss terms using multi-task learning. Gradient projection reconciled gradients from different loss items. Our experiments on 2D and 3D geometries demonstrate that 3DMeshNet can effectively balance meshing overhead and mesh quality, producing high-quality meshes for complex shapes where PINN methods might struggle to maintain mesh quality. 3DMeshNet produces mesh quality that surpasses existing approaches, achieving a meshing overhead reduction of 4 to 8 times compared to TFI methods and decreasing training times by up to 85% relative to other neural network-based methods. To the best of our knowledge, this is the first work to introduce PINNs for 3D mesh generation.

The remainder of this paper is organized as follows: Section 2 reviews related work on mesh generation methods. Section 3 provides a detailed description of the proposed 3DMeshNet method. Section 4 presents experimental results comparing different methods on two 2D geometries and four 3D geometries. Lastly, Section 5 concludes the paper and discusses future work.

## 2. Related work

Structured meshes provide simplicity, making them widely used in numerous simulations that necessitate precise alignment of elements as dictated by analytical requirements [16]. The demand for precision has also stimulated the development of mesh generation methods [7,8]. The methods for generating structured meshes include conformal mapping, algebraic methods, and PDE methods [17]. Algebraic methods typically employ various interpolation techniques or special functions for mesh creation [18]. Conformal mapping, which employs angle-preserving transformations derived from complex function theory, facilitates the generation of 2D meshes by altering the computational space. However, expanding these methods to 3D mesh generation presents significant

challenges. PDE methods, on the other hand, generate meshes by solving elliptic, parabolic, or hyperbolic equations within a specified transformation domain [19]. Although the algebraic method offers simplicity and rapid mesh creation, it may lead to mesh degradation in areas of complex geometry or cause mesh elements to overlap or breach boundaries. Structured meshes generated using the differential equation method are of high quality but require high computational effort.

Artificial intelligence represented by neural networks has been vigorously developed in recent years [20,21]. Neural networks can autonomously learn artificial experiences and fit objective functions from high-dimensional parameter spaces, and significant progress has been made in knowledge-based physical problems during the last several years [22,23]. Obiols-Sales O et al. [11] proposed that CFD-Net combines physical simulation and deep learning in a coupled framework to accelerate the convergence of Reynolds Averaged Navier–Stokes simulations. Neural networks are also trained to learn the lift coefficients of airfoils with various shapes under different flow Mach numbers, Reynolds numbers, and diverse angles of attack [22]. Gholami et al. [23] used neural networks along with numerical simulation to estimate flow depth variables and velocity for typical cases such as a 60° bend in a tube. The use of neural networks to solve PDEs has become increasingly prevalent and impactful in academic research. The PINNs designed by Raissi M [24] et al. employ a DNN to approximate the solutions of PDEs, embedding the residuals that govern the PDE and its initial/boundary conditions within the loss function. Various variants of PINNs have been proposed to enhance their accuracy and efficiency. CanPINN [25] combines automatic and numerical differentiation in PINNs to improve training efficiency and accuracy. Meanwhile, ATPINNs [26] address the issues of low accuracy and non-convergence in original PINNs via auxiliary-task learning. These variants aim to strengthen PINN's performance in solving PDEs.

In addition to applying artificial intelligence in physics for solving PDEs, some researchers have explored incorporating artificial intelligence into mesh generation, mesh quality discrimination, and other areas to overcome the limitations of automation and intelligence in traditional mesh methods. Chen [3] presented a neural network-based mesh quality indicator for 3D cylinder meshes, along with a benchmark dataset. Wang [27] developed an algorithm to convert mesh data into graph data, subsequently introducing a deep graph neural network, GMeshNet, for mesh quality evaluation. MQNet [28] is introduced as a structured mesh quality evaluation network that leverages dynamic graph attention, treating mesh evaluation as a graph classification task. Yilmaz and Kuzuoglu [29] proposed a particle swarm optimization algorithm aimed at optimizing hexahedral mesh quality, addressing the inefficiencies of the Laplace mesh smoothing algorithm in concave areas. Gargallo-Peiró A [30] suggested an optimization method for triangular and quadrilateral meshes on parameterized surfaces, focusing on node relocation to enhance mesh quality and prevent element inversion. Data-driven approaches to mesh generation have seen various implementations. Lowther et al. [31] applied neural networks to mesh density prediction in finite element mesh adaptation, using density information provided by the neural network to determine the size and placement of elements. MeshingNet [32] leverages posterior error estimates on an initial coarse mesh to predict non-uniform mesh densities for refinement. However, generating the training dataset is costly, and the model targets two-dimensional unstructured meshes. Huang et al. [33] developed a convolutional neural network to predict optimal mesh densities for diverse geometries. The validation phase required more than 60,000 simulations, supported by a training dataset of 20,000 simulations. Such extensive data demands may present challenges regarding computational resources and time allocation. Alexis Papagiannopoulos [34] introduced a neural network scheme for creating 2D simplicial meshes, using networks trained on meshed contour data to approximate vertex count, placement, and connectivity, though requiring extensive datasets for contours with a

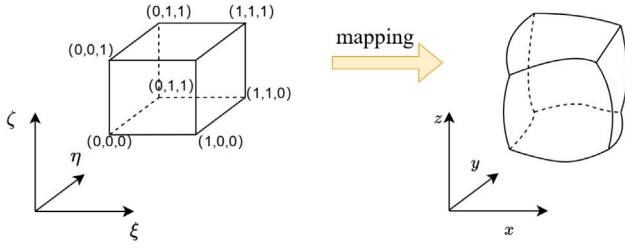


Fig. 1. The unit cube is mapped to a curved hexahedron.

high number of vertices. Peng et al. [35] employed two Artificial Neural Networks (ANNs) trained on an initial hybrid grid to predict point advancement and control grid sizing, with the ANN-based Advancing Layer Method generating initial training grids. Kim et al. [36] utilized Graph Convolutional Neural Networks to learn local error densities and predict target mesh edge lengths, generating 16,000 control and feature meshes for training. However, these supervised learning meshing techniques necessitate significant effort in producing large training datasets, complicated by the costly creation and processing of these datasets, particularly for meshes with complex geometries or high vertex counts. Moreover, generating training data demands domain-specific expertise. Physics-informed methods utilizing unsupervised learning to minimize the need for extensive training data in mesh generation have emerged. MGNet [8] proposed an approach for structured mesh creation, which is the first application of PINNs in this field. The network is designed to unsupervisedly learn meshing rules between parametric and computational spaces, requiring only boundary specifications. Nevertheless, MGNet faces challenges like training instability and extended durations. Chen [7] enhanced mesh quality by modifying loss terms with an auxiliary line strategy, initially requiring the manual selection of these lines as ground truth. However, it is still oriented to a 2D problem and requires additional manual intervention for drawing auxiliary lines. MeshNet first generates a coarse mesh using the traditional method and then adds the points of the coarse mesh as data items inside the training of the PINN, improving the quality of the generated mesh [37]. However, it introduces additional overhead for computation and training. While physics-informed methods alleviate the need for extensive manual data, they encounter issues such as slow training speeds, unstable convergence, and long training times. Furthermore, most of these approaches focus on 2D mesh generation, leaving the use of artificial intelligence methods for 3D structured mesh generation as an open problem.

### 3. 3DMeshNet: A three-dimensional differential neural network for structured mesh generation

#### 3.1. Problem setup

The task of structured mesh generation involves applying a potential mapping from the parametric space to the computational space for a given geometry. As shown in Fig. 1, the unit cube  $0 \leq \xi \leq 1, 0 \leq \eta \leq 1, 0 \leq \zeta \leq 1$  (left) is mapped to the computational space (right) in Fig. 1. Typically, the parametric space, represented by the coordinates  $(\xi, \eta, \zeta)$ , is defined as a basic cube  $[0,1] \times [0,1] \times [0,1]$ , with each edge oriented either horizontally or vertically. The primary goal of any structured mesh generation technique is to create a mapping  $(\xi, \eta, \zeta)$  to  $(x, y, z)$  between these two spaces. This mapping should be unique and smooth, ensuring that the resulting mesh accurately conforms to the geometry's boundaries that require discretization.

In numerical computing, algebraic methods [38] and PDE [39] methods are the two most commonly used structured mesh generation techniques. Algebraic methods use algebraic interpolation to

describe the potential mapping relationship between the parametric space  $(\xi, \eta, \zeta)$  and the computational space  $(x, y, z)$ .

The generation of meshes in 3D spaces through PDEs involves determining the correspondence between node coordinates in computational and parametric spaces. This is achieved by treating the process as a boundary-value problem solved by a system of elliptic PDEs. In this approach, elliptic equations are employed for 3D mesh generation. The process first solves for parametric coordinates within the parametric space using elliptic mesh generation methods. Subsequently, these parametric meshes are transformed back into the computational space, resulting in the acquisition of the computational surface mesh coordinates.

The governing differential equation is of the form of

$$\nabla^2 \xi^i = P^i \quad (i = 1, 2, 3), \quad (1)$$

or

$$\begin{cases} \xi_{xx} + \xi_{yy} + \xi_{zz} = P(\xi, \eta, \zeta) \\ \eta_{xx} + \eta_{yy} + \eta_{zz} = Q(\xi, \eta, \zeta) \\ \zeta_{xx} + \zeta_{yy} + \zeta_{zz} = R(\xi, \eta, \zeta), \end{cases} \quad (2)$$

where  $\nabla^2$  denotes the Laplacian operator in Cartesian coordinate system,  $P^i = P^i(\xi^1, \xi^2, \xi^3)$ ,  $P^1 = P$ ,  $P^2 = Q$ , and  $P^3 = R$  are referred to as source terms. The source terms  $P$ ,  $Q$ , and  $R$  are helpful in stretching the mesh surface  $(\xi, \eta, \zeta)$  respectively. Negative values of the source terms cause the corresponding mesh faces to move toward the decreasing curve coordinates, while positive values of the source terms have the opposite effect. Therefore, the mesh density can be adjusted by adjusting the values of the source terms.

#### 3.2. The architecture of 3DMeshNet

We propose 3DMeshNet, a novel neural network-based approach for structured 3D mesh generation. It leverages the representation learning capabilities of neural networks to learn the rules governing mesh partitioning and determine the mapping between parametric and computational space by encoding point coordinates through a deep network. Specifically, the network mesh partitioning rules by minimizing a weighted loss function comprising the residuals of governing differential equations and surface constraint. Once trained, 3DMeshNet can generate meshes of arbitrarily specified sizes for the corresponding geometries in a feedforward.

The architecture of 3DMeshNet is illustrated in Fig. 2. It comprises a neural network and a physics-informed learning part. The neural network component primarily learns the rules of mesh partitioning, while the physics-informed learning part provides physical information from the elliptic governing differential equations.

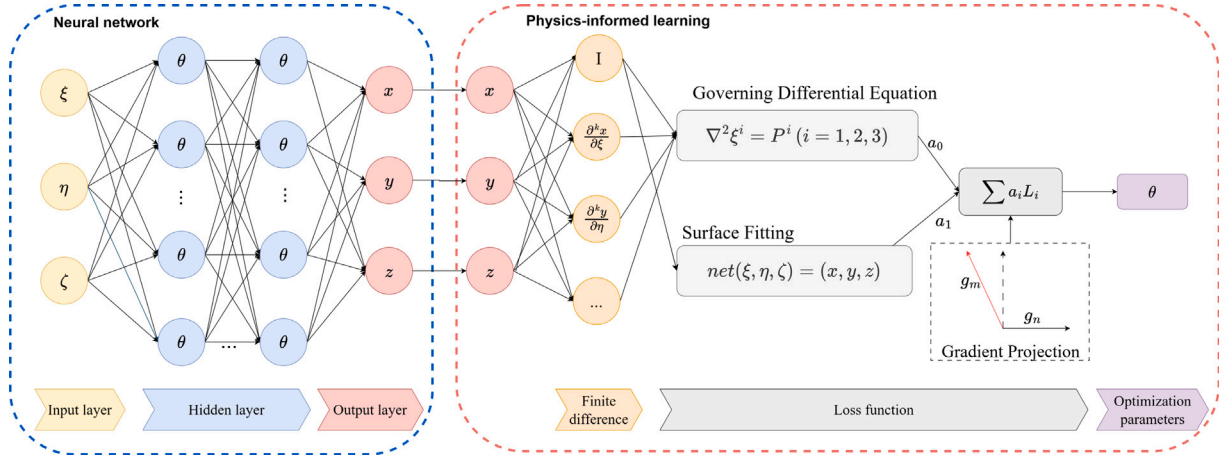
The input to 3DMeshNet is sampled points from the interior and surface points of the parametric space. Each input comprises a set of 3D points  $\{(\xi_i, \eta_i, \zeta_i)\}_{i=1}^{i=N}$ , where  $N$  is the number of points. The network output consists of points  $\{(x_i, y_i, z_i)\}_{i=1}^{i=N}$  representing the mesh in the computational space.

The neural network part contains an input, multiple hidden layers, and an output layer. The input layer takes training points  $(\xi, \eta, \zeta)$  and passes them to hidden layers. Within hidden layers, the network performs computations to extract high-dimensional features via Eq. (3):

$$\chi^k = \sigma(\chi^{k-1} W^{k-1} + b^{k-1}), \quad (3)$$

where  $\chi$  denotes the output of a hidden layer,  $\sigma$  is the activation function, and  $W$  and  $b$  are the layer weights and bias, respectively. Subsequently, the output layer generates predictions for the computational space coordinates based on the received hidden representations.

In the Physics-informed learning part, elliptic control equations are employed to guide the Neural network's training in learning mesh partitioning rules, with boundary conditions describing the surface fitting of



**Fig. 2.** The architecture of the proposed 3DMeshNet. It fuses a Neural network, which learns mesh partitioning from 3D points, with a Physics-informed learning part that enforces physical laws via elliptic differential equations. The Neural network extracts features and predicts mesh coordinates, while the Physics-informed learning part uses a finite difference layer for efficient derivatives calculation, guiding the NN's training.

the given 3D object. In the Finite Difference layer, 3DMeshNet applies the Finite Difference method, based on the Taylor series expansion, to improve efficiency in derivative computation. The loss function is determined through Loss Function Reweighting, informed by the 3D elliptical equation and Surface Fitting. Loss Function Reweighting employs a multi-task learning strategy and surface point weighting scheme to reweight the two-part loss. The multi-task learning strategy leverages homoscedastic uncertainty principles from Bayesian inference to adjust weights  $a$  according to task-specific uncertainties. The surface point weighting scheme assigns weights based on the Euclidean distance between predicted and actual surface points, thereby enhancing the quality of the boundary mesh. Moreover, gradient projection effectively mitigates gradient conflict between the elliptic equation gradient and the Surface Fitting gradient. Lastly, network parameters are then updated across training sessions through loss backpropagation.

### 3.3. Loss function with finite difference

The loss function of 3DMeshNet consists of two parts: a boundary loss for surface fitting and a 3D elliptic PDE system as basic governing differential equations. The 3D elliptic PDE provides the physical-informed learning aspect for mesh generation. By embedding the 3D elliptic PDE into the loss function, we aim to minimize it during the unsupervised learning process. Since 3DMeshNet learns a mapping from the parametric space  $(\xi, \eta, \zeta)$  to the computational space  $(x, y, z)$ , we treat  $(\xi, \eta, \zeta)$  as the input and  $(x, y, z)$  as the output of the network. Therefore, we cannot directly embed Eq. (2) into the loss function. Instead, we use the equivalent form of Eq. (4) embedded into the loss function.

$$\begin{aligned}
 &\alpha_1 x_{\xi\xi} + \alpha_2 x_{\eta\eta} + \alpha_3 x_{\zeta\zeta} + 2\beta_{12} x_{\xi\eta} + 2\beta_{23} x_{\eta\zeta} + 2\beta_{31} x_{\zeta\xi} = 0 \\
 &\alpha_1 y_{\xi\xi} + \alpha_2 y_{\eta\eta} + \alpha_3 y_{\zeta\zeta} + 2\beta_{12} y_{\xi\eta} + 2\beta_{23} y_{\eta\zeta} + 2\beta_{31} y_{\zeta\xi} = 0 \\
 &\alpha_1 z_{\xi\xi} + \alpha_2 z_{\eta\eta} + \alpha_3 z_{\zeta\zeta} + 2\beta_{12} z_{\xi\eta} + 2\beta_{23} z_{\eta\zeta} + 2\beta_{31} z_{\zeta\xi} = 0 \\
 &\alpha_1 = (x_{\eta}^2 + y_{\eta}^2 + z_{\eta}^2)(x_{\xi}^2 + y_{\xi}^2 + z_{\xi}^2) - (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta}), \\
 &\alpha_2 = (x_{\xi}^2 + y_{\xi}^2 + z_{\xi}^2)(x_{\eta}^2 + y_{\eta}^2 + z_{\eta}^2) - (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta}), \\
 &\alpha_3 = (x_{\xi}^2 + y_{\xi}^2 + z_{\xi}^2)(x_{\eta}^2 + y_{\eta}^2 + z_{\eta}^2) - (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta}), \\
 &\beta_{12} = (x_{\eta}x_{\xi} + y_{\eta}y_{\xi} + z_{\eta}z_{\xi})(x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta}) \\
 &\quad - (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta})(x_{\xi}^2 + y_{\xi}^2 + z_{\xi}^2), \\
 &\beta_{23} = (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta})(x_{\eta}x_{\xi} + y_{\eta}y_{\xi} + z_{\eta}z_{\xi}) \\
 &\quad - (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta})(x_{\eta}^2 + y_{\eta}^2 + z_{\eta}^2), \\
 &\beta_{31} = (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta})(x_{\eta}x_{\xi} + y_{\eta}y_{\xi} + z_{\eta}z_{\xi}) \\
 &\quad - (x_{\xi}x_{\eta} + y_{\xi}y_{\eta} + z_{\xi}z_{\eta})(x_{\eta}^2 + y_{\eta}^2 + z_{\eta}^2),
 \end{aligned} \tag{4}$$

where  $x_{\xi}$  represents the first-order partial derivative of  $x$  with respect to  $\xi$ , while  $x_{\xi\eta}$  signifies the second-order partial derivative of  $x$ , taken sequentially first with respect to  $\xi$  and then  $\eta$ . This notation is similarly extended to other second-order derivatives. The detailed derivation of the above equation can be found in [40]. We can rewrite the above equation in a more compact form as a double inner product of two tensors. First, we can define  $\mathbf{u}$  as follows:

$$\mathbf{u} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}. \tag{6}$$

The second-order derivative tensor,  $\mathbf{D}$ , contains each second partial derivative of the components of  $\mathbf{u}$  with respect to  $\xi, \eta$ , and  $\zeta$ .

$$\mathbf{D} = \begin{pmatrix} \frac{\partial^2 \mathbf{u}}{\partial \xi^2} & \frac{\partial^2 \mathbf{u}}{\partial \xi \partial \eta} & \frac{\partial^2 \mathbf{u}}{\partial \xi \partial \zeta} \\ \frac{\partial^2 \mathbf{u}}{\partial \xi \partial \eta} & \frac{\partial^2 \mathbf{u}}{\partial \eta^2} & \frac{\partial^2 \mathbf{u}}{\partial \eta \partial \zeta} \\ \frac{\partial^2 \mathbf{u}}{\partial \xi \partial \zeta} & \frac{\partial^2 \mathbf{u}}{\partial \eta \partial \zeta} & \frac{\partial^2 \mathbf{u}}{\partial \zeta^2} \end{pmatrix}. \tag{7}$$

The coefficient tensor,  $\mathbf{A}$ , organizes the coefficients  $\alpha_i$  and  $\beta_{ij}$  into matrix form.

$$\mathbf{A} = \begin{pmatrix} \alpha_1 & \beta_{12} & \beta_{31} \\ \beta_{12} & \alpha_2 & \beta_{23} \\ \beta_{31} & \beta_{23} & \alpha_3 \end{pmatrix}. \tag{8}$$

Then the loss associated with the governing differential equations can be expressed as:

$$\text{Loss}_1 = \frac{1}{N_r} \sum_{n=1}^{N_r} |\mathbf{A} : \mathbf{D}(\mathbf{u})|^2, \tag{9}$$

where  $:$  denotes the double contraction (or double inner product) between tensors  $\mathbf{A}$  and  $\mathbf{D}$ . In this case, this operation sums the products of corresponding elements in  $\mathbf{A}$  and  $\mathbf{D}$ . In addition, the loss function with the geometric surface constraint can be written as:

$$\text{Loss}_2 = \frac{1}{N_b} \sum_{n=1}^{N_b} |(x_n, y_n, z_n) - f(\xi_n, \eta_n, \zeta_n)|^2. \tag{10}$$

The loss function comprises two parts: the first part,  $\text{Loss}_1$ , is the control equation loss term, and  $\text{Loss}_2$  is the geometric surface constraint term. Here,  $N_r$  and  $N_b$  represent the number of training points for the control equation and the geometric surface.  $(x_n, y_n, z_n)$  denotes the coordinates of the  $n$ th actual surface point, and  $f(\xi_n, \eta_n, \zeta_n)$  refers to the predicted value corresponding to the input at the surface point.

To achieve a more efficient implementation, we do not use automatic differentiation for derivative calculations because it is time-consuming [25]. Instead, we shift to using the finite differences method

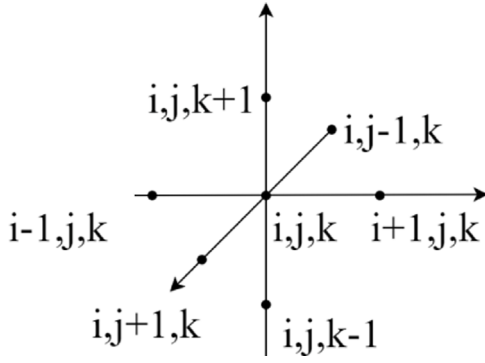


Fig. 3. Coordinate points of the 3D finite difference.

for calculating derivatives. The finite differences method is based on the Taylor series expansion, allowing us to compute derivatives more efficiently and effectively.

As shown in Fig. 3, the central point  $(i, j, k)$  serves as the reference point, with its neighboring points in the three orthogonal directions labeled as  $(i+1, j, k)$ ,  $(i-1, j, k)$ ,  $(i, j+1, k)$ ,  $(i, j-1, k)$ ,  $(i, j, k+1)$ , and  $(i, j, k-1)$ . The relative positions of these points to the central point represent the layout of points in the finite differences framework used for calculating partial derivatives. Combining the function values at these points with the function value at the central point and applying the Taylor expansion, one can approximate the partial derivatives at the central point. Specifically, the derivation equation for the finite differences method is as follows:

$$\begin{aligned} \mathbf{u}_\xi &= (\mathbf{u}_{i+1,j,k} - \mathbf{u}_{i-1,j,k}) / 2h_1, \\ \mathbf{u}_\eta &= (\mathbf{u}_{i,j+1,k} - \mathbf{u}_{i,j-1,k}) / 2h_2, \end{aligned} \quad (11)$$

$$\begin{aligned} \mathbf{u}_\zeta &= (\mathbf{u}_{i,j,k+1} - \mathbf{u}_{i,j,k-1}) / 2h_3, \\ \mathbf{u}_{\xi\xi} &= (\mathbf{u}_{i+1,j,k} - 2\mathbf{u}_{i,j,k} + \mathbf{u}_{i-1,j,k}) / h_1^2, \\ \mathbf{u}_{\eta\eta} &= (\mathbf{u}_{i,j+1,k} - 2\mathbf{u}_{i,j,k} + \mathbf{u}_{i,j-1,k}) / h_2^2, \\ \mathbf{u}_{\zeta\zeta} &= (\mathbf{u}_{i,j,k+1} - 2\mathbf{u}_{i,j,k} + \mathbf{u}_{i,j,k-1}) / h_3^2. \end{aligned} \quad (12)$$

$$\begin{aligned} \mathbf{u}_{\xi\eta} &= \frac{1}{4h_1h_2} (\mathbf{u}_{i+1,j+1,k} - \mathbf{u}_{i+1,j-1,k} \\ &\quad - \mathbf{u}_{i-1,j+1,k} + \mathbf{u}_{i-1,j-1,k}), \\ \mathbf{u}_{\eta\zeta} &= \frac{1}{4h_2h_3} (\mathbf{u}_{i,j+1,k+1} - \mathbf{u}_{i,j+1,k-1} \\ &\quad - \mathbf{u}_{i,j-1,k+1} + \mathbf{u}_{i,j-1,k-1}), \\ \mathbf{u}_{\xi\zeta} &= \frac{1}{4h_3h_1} (\mathbf{u}_{i+1,j,k+1} - \mathbf{u}_{i+1,j,k-1} \\ &\quad - \mathbf{u}_{i-1,j,k+1} + \mathbf{u}_{i-1,j,k-1}), \end{aligned} \quad (13)$$

where  $h_1, h_2, h_3$  represent the steps in the three spatial directions.

### 3.4. Loss function reweighting

As the loss function consists of two components, namely the Governing Differential Equation and the Surface Fitting Loss, we can consider these two optimization objectives as a multi-task optimization problem. Moreover, a significant numerical difference exists between the losses of these two parts. Weight hyper-parameters are expensive to tune and often take time for each trial. Therefore, it is desirable to find a more convenient approach that can learn the optimal weights. Consequently, We incorporate a strategy for multi-task learning as presented in source [41] to assign weight appropriately to the losses of distinct tasks.

The methodology utilizes homoscedastic uncertainty principles from Bayesian inference to assess and articulate the variance in confidence across tasks, enabling a sophisticated approach to loss weighting based on task-specific uncertainties. Let  $NN(X; \theta)$  denote the output of a

neural network with input  $x$  and  $u$  is the model's output, in this paper represented as  $(x, y, z)$ . We define the following probabilistic model:

$$p(u|NN(X; \theta)) = \mathcal{N}(NN(X; \theta), a^2), \quad (14)$$

and Eq. (14) describes the conditional probability distribution of the target variable  $u$  given the input  $X$  and the neural network prediction  $NN(X; \theta)$ . We assume that the distribution of the target variable  $u$  follows a Gaussian distribution with mean equal to the predicted value  $NN(X; \theta)$  and variance  $a^2$ . With the scalar  $a$  as a noise parameter. In a setting of a neural network with  $K$  tasks, the multi-objective likelihood can be written as:

$$\begin{aligned} p(u_1, \dots, u_K | NN(X; \theta)) \\ = p(u_1 | NN(X; \theta)) \cdots p(u_K | NN(X; \theta)). \end{aligned} \quad (15)$$

In maximum likelihood estimation, it is often convenient to work with the natural logarithm of the likelihood function, the log likelihood can be written as:

$$\begin{aligned} \log p(u | NN(X; \theta)) \\ = \log \left[ (2\pi a^2)^{-\frac{1}{2}} \exp \left( -\frac{1}{2a^2} \|u - NN(X; \theta)\|^2 \right) \right] \\ = \log (2\pi a^2)^{-\frac{1}{2}} + \log \left[ \exp \left( -\frac{1}{2a^2} \|u - NN(X; \theta)\|^2 \right) \right] \end{aligned} \quad (16)$$

$$\begin{aligned} &= -\frac{1}{2} \log (2\pi a^2) - \frac{1}{2a^2} \|u - NN(X; \theta)\|^2 \\ &= -\frac{1}{2} \log (2\pi) - \frac{1}{2} \log (a^2) - \frac{1}{2a^2} \|u - NN(X; \theta)\|^2, \\ \log p(u | NN(X; \theta)) &\propto -\frac{1}{2a^2} \|u - NN(X; \theta)\|^2 - \log a, \end{aligned} \quad (17)$$

and we can therefore maximize:

$$\begin{aligned} \log p(u_1, \dots, u_K | NN(X; \theta)) \\ = \log p(u_1 | NN(X; \theta)) + \cdots + \log p(u_K | NN(X; \theta)) \\ = \log \mathcal{N}(u_1; NN(X; \theta), a_1^2) + \cdots + \log \mathcal{N}(u_K; NN(X; \theta), a_K^2) \\ = \left( -\frac{1}{2a_1^2} L_1 - \log a_1 \right) + \cdots + \left( -\frac{1}{2a_K^2} L_K - \log a_K \right) \\ = \left( -\frac{1}{2a_1^2} L_1 - \frac{1}{2} \log a_1^2 \right) + \cdots + \left( -\frac{1}{2a_K^2} L_K - \frac{1}{2} \log a_K^2 \right) \\ \propto \left( -\frac{1}{2a_1^2} L_1 - \log(1 + a_1^2) \right) + \cdots + \left( -\frac{1}{2a_K^2} L_K - \log(1 + a_K^2) \right). \end{aligned} \quad (18)$$

According to Eq. (18), to facilitate optimization, we prefer to maximize the sum of the logarithms of these likelihoods, which simplifies to a sum of terms involving task-specific losses  $L_i$  and noise parameters  $a_i$ . The logarithm of  $a^2$  is modified to  $\log(1 + a^2)$ , ensuring the expression remains positive and avoids training issues where the loss might otherwise become negative. This leads to our minimization target:

$$L(\theta, a_1, \dots, a_K) = \sum_i^K \left( \frac{1}{2a_i^2} L_i + \log(1 + a_i^2) \right), \quad (19)$$

this objective function balances each task's loss against its uncertainty, represented by  $a$ , now a parameter to be learned alongside the neural network's weights. It allows adaptive learning of each task's importance, preventing the network from discounting easier tasks with smaller uncertainties. Additionally, the term  $\log(1 + a_i^2)$  acts as a regularization, averting excessively large values of  $a$  and ensuring a balanced task weighting within the multi-task learning framework.

Another innovation in Loss Function Reweighting is the implementation of a surface point weighting scheme. This scheme emphasizes the importance of accurately capturing surface conditions by assigning higher weights to poorly predicted surface points and lower weights to well-predicted ones. As a result, 3DMeshNet can more effectively learn and conform to surface conditions, enhancing the mesh quality at the boundaries.

$$weight = \left\| f(\xi_n, \eta_n, \zeta_n) - (x_n, y_n, z_n) \right\| + 1. \quad (20)$$

We calculate the Euclidean distance between the predicted and actual surface points according to Eq. (20). The distance is then incremented by 1 to obtain the weight,  $w = \text{distance} + 1$ , or the distance can be used directly as the weight,  $w = \text{distance}$ . In practice, multiplying the weight by  $\text{Loss}_2$  results in  $|(x_n, y_n, z_n) - f(\xi_n, \eta_n, \zeta_n)|^3$ . Here, we draw inspiration from the work of Wang et al. [42], who proposed applying different power operations to each loss term. This ensures that all terms are approximately the same order of magnitude, allowing them to be optimized synchronously during training. Motivated by their work, we applied a similar weighting strategy to improve the loss function. However, we fixed the exponent for the loss terms to reduce training overhead and still achieve good training results. After introducing the loss function adaptive weighting strategy as well as the surface point weighting mechanism, the loss function of 3DMeshNet can be rewritten in the following form:

$$\text{Loss}(x, y, z) = \frac{1}{2a_1^2} \text{Loss}_1 + \frac{1}{2a_2^2} \text{weight} \cdot \text{Loss}_2 + \log(1 + a_1^2 + a_2^2). \quad (21)$$

Finally, 3DMeshNet employs Eq. (21) as the loss function to guide the network's training.

### 3.5. Gradient projection

In addition to balancing the Surface Fitting loss with the Governing Differential Equation loss efficiently, our approach introduces gradient projection to address the issue of ill-conditioned gradients from a gradient perspective. Previous works, such as MGNet [8] and MeshNet [32], have focused primarily on numerical improvements to the loss function, with little exploration into enhancements from a gradient standpoint. Furthermore, upon examining the loss function, it is evident that substantial oscillations in the loss are not conducive to the optimal collection of structured mesh results. Therefore, we have incorporated the gradient projection mechanism to improve network convergence. This novel methodology offers a comprehensive solution by tackling numerical and gradient aspects, enhancing the overall efficacy of structured mesh generation.

We define  $\varphi_{mn}$  as the angle between two task gradients  $g_m$  and  $g_n$ , and when  $\cos \varphi_{mn} < 0$  it means that the gradients are conflicting. Specifically, mitigating the gradient conflict of the loss function can include the following steps:

(1) Assume that the gradient of task  $T_m$  is  $g_m$  and the gradient of task  $T_n$  is  $g_n$ .

(2) Determine whether  $g_m$  conflicts with  $g_n$  by calculating the cosine similarity between the vectors  $g_m$  and  $g_n$ , where a negative value indicates the gradient of the conflict. The cosine similarity is calculated as follows.

$$\cos \varphi_{mn} = \frac{g_m \cdot g_n}{\|g_m\| \times \|g_n\|}. \quad (22)$$

(3) If the cosine similarity is negative, replace  $g_n$  with its projection  $g_n^{PC}$  on the normal plane of  $g_m$ , that is,  $g_n^{PC} = g_m^{PC} - \frac{g_m^{PC} \cdot g_n}{\|g_m^{PC}\|^2} g_n$ . If the gradients are not conflicting,  $\cos \varphi_{mn} > 0$  is non-negative, and the original gradient  $g_m$  is kept constant.

Fig. 4 illustrates the principle of gradient conflict resolution. Fig. 4(a) shows that the gradient projection algorithm does not change the gradients of non-conflicting tasks. Fig. 4(b) demonstrates that tasks  $m$  and  $n$  have conflicting gradient directions. In Fig. 4(c), the gradient projection algorithm for conflicting gradients is shown, where the gradient of task  $n$  is projected onto the normal vector of the gradient of task  $m$ .

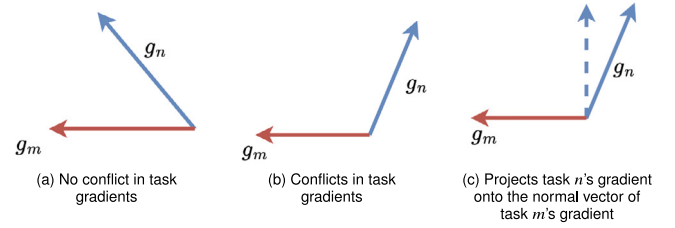


Fig. 4. Conflicting gradients and gradient projections.

## 4. Experimental results

In this section, we demonstrate the effectiveness of the proposed 3DMeshNet in both 2D and 3D mesh generation cases. In the 2D experiments, we use a configuration with hidden layer sizes of 45, 45, 45; for the 3D experiments, we use a configuration with 85, 85, 85, 85 hidden layer sizes. Tanh serves as the activation function in all cases. Due to data dimension limitations, we opt for a lightweight network model to extract features instead of employing larger-scale models. 3DMeshNet randomly samples 4000 internal points and 6000 surface points from the parametric space for training during each training iteration, and it employs Adam and L-BFGS-B optimizers to refine network parameters. Specifically, 3DMeshNet first employs the Adam optimizer for 12,000 iterations of training. The learning rate is  $1e-3$ , and it is reduced by a factor of 0.9 every 1000 epochs. Subsequently, the L-BFGS-B is employed to optimize the model parameters further. All experiments are implemented using PyTorch. Although the training processes are conducted on the 3080Ti GPU, due to the lightweight network model, the meshing process can be performed rapidly on CPU platforms. To validate the effectiveness of 3DMeshNet, we compared it with TFI [38] and PDE [39]. As NN-based models, we also selected PINN [24] and MGNet [8] to compare the quality of the generated structured meshes with 3DMeshNet.

In addition, we need to supplement some details regarding the experimental setup for the comparison methods. For the TFI and PDE methods, we followed the principles outlined in [40] and implemented the corresponding Python code.

For the NN-based models in the 2D experiments, the PINN uses the same network structure as 3DMeshNet. It has two input neurons to receive  $\xi$  and  $\eta$ , and two output neurons to predict the 2D mesh coordinates  $x$  and  $y$ . The loss function is the sum of  $\text{Loss}_1$  and  $\text{Loss}_2$ , but without the three mechanisms introduced in this paper. In the 3D experiments, PINN and 3DMeshNet share the same network structure. There are three input neurons to receive  $\xi$ ,  $\eta$ , and  $\zeta$ , and three output neurons to predict the 3D mesh coordinates  $x$ ,  $y$ , and  $z$ . The loss function is again the sum of  $\text{Loss}_1$  and  $\text{Loss}_2$ , but without the three mechanisms introduced in this paper.

For the MGNet method, we followed the original network setup described in the paper. In the 2D experiments, two networks are used to predict the  $x$  and  $y$  coordinates of the mesh. According to the structure of MGNet, the number of input neurons is 4 (since there are two sub-networks, each receiving  $\xi$  and  $\eta$  as inputs), and there are two output neurons, predicting the 2D coordinates  $x$  and  $y$ . Since MGNet is originally designed for 2D structured mesh generation, it cannot be directly applied to 3D mesh generation without modification. Therefore, we extended MGNet by using three sub-networks to predict the  $x$ ,  $y$ , and  $z$  coordinates of the 3D mesh. In this setup, the number of input neurons is 6, since each of the three sub-networks receives  $\xi$ ,  $\eta$ , and  $\zeta$ . Each sub-network has one output neuron, resulting in a total of three output neurons that produce the final  $x$ ,  $y$ , and  $z$  coordinates. The number of hidden layers in each sub-network follows the configuration of 3DMeshNet. Additionally, we retain the design of the “Excitation Block” and “Penalty Coefficient” as described in the original paper.

**Table 1**

Numerical results comparison of structured mesh generation on 2D Case 1 using TFI, PDE, PINN, MGNet, and 3DMeshNet.

| Method    | Avg. test set loss↓                       | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Mesh validity |
|-----------|-------------------------------------------|------------------------------|------------------------------|---------------------|----------------|---------------|
| TFI       | –                                         | 75.61654                     | 104.53220                    | 0.16243             | 4.91350        | Y             |
| PDE       | –                                         | 74.72664                     | 105.50302                    | 0.17303             | 4.29008        | Y             |
| PINN      | $5.0700 \times 10^{-6}$                   | 75.86799                     | 104.10900                    | 0.16059             | 4.52362        | Y             |
| MGNet     | $9.5063 \times 10^{-7}$                   | 75.99211                     | 104.15644                    | 0.15853             | <b>4.24366</b> | Y             |
| 3DMeshNet | <b><math>3.4100 \times 10^{-7}</math></b> | <b>76.31713</b>              | <b>103.61675</b>             | <b>0.15812</b>      | 4.24556        | Y             |

**Table 2**

Comparison of numerical results for structured mesh generation on the 2D Case 2 using TFI, PDE, PINN, MGNet, and 3DMeshNet.

| Method    | Avg. test set loss↓ | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Mesh validity |
|-----------|---------------------|------------------------------|------------------------------|---------------------|----------------|---------------|
| TFI       | –                   | 61.38721                     | 118.6771                     | 0.31882             | 2.26686        | Y             |
| PDE       | –                   | 74.80033                     | 105.2497                     | 0.16962             | 2.03112        | Y             |
| PINN      | 0.00400             | <b>82.13464</b>              | <b>97.87325</b>              | <b>0.08773</b>      | <b>1.05901</b> | N             |
| MGNet     | 0.00398             | 81.73544                     | 98.30392                     | 0.09252             | 2.06594        | Y             |
| 3DMeshNet | <b>0.00040</b>      | 81.72149                     | 98.32237                     | 0.09268             | 2.07669        | Y             |

In assessing the generated mesh quality, we employ the loss on the test set as an auxiliary indicator of the mesh quality. Additionally, we utilize a comprehensive evaluation using the following five criteria for mesh quality to determine the overall integrity of the generated mesh:

**(1) Average Test Set Loss (2D, 3D):** For the test set, we manually generate high-quality structured meshes for each shape, with the same mesh scale as the visualized mesh. The final test set loss is computed as the mean squared error between the meshes generated by 3DMeshNet, PINN, and MGNet, and these manually pre-generated high-quality structured meshes.

**(2) Average Minimum Included Angle and Average Maximum Included Angle (2D, 3D):** Both Average Minimum Included Angle and Average Maximum Included Angle are measures of mesh skewness. For all types of meshes, the Minimum Included Angle represents the smallest angle in a mesh, while the Maximum Included Angle represents the largest angle. The Average Maximum Included Angle is calculated by summing the Maximum Included Angles of all cells and dividing by the total number of cells. Similarly, the Average Minimum Included Angle is calculated by summing the Minimum Included Angles of all cells and dividing by the total number of cells.

**(3) Equiangle Skewness (2D, 3D):** Equiangle Skewness is expressed as the maximum ratio of the angle of the mesh to the angle of the equiangular mesh. The angle skewness varies between 0 (good) and 1 (poor).

**(4) Aspect Ratio (2D, 3D):** The hexahedral cell aspect ratio is computed from the ratio of the maximum length, width, and height and the minimum length, width, and height. The aspect ratio is always greater than or equal to 1, with a value of 1 representing a cube.

**(5) Centroid Skewness (3D):** Centroid Skewness is one minus the minimum dot product between the cell face normal and the vector connecting the cell centroid and the face centroid. Values range from 0 (no-skew) to 1 (collapsed cell). This measure is only valid for block cells.

**(6) Cell Non-Orthogonality (3D):** This measure operates on pairs of neighboring cells that share a face. An angle is calculated between a line connecting the two cells' centroids and the shared face's normal. The maximum value of this measure for all faces of a given cell is reported per cell. The angle shown as  $\theta$  in Fig. 5.

**(7) Meshing Overhead (2D, 3D):** This metric measures the time required to generate a mesh on a given geometry, using seconds (s) as the unit of measurement.

**(8) Mesh validity (2D, 3D):** This metric assesses the efficacy of the mesh generated on the geometry using the respective method. Meshes are deemed ineffective if there are notable instances of elements extending beyond boundaries or lying below the boundary shape. Numerically, in this paper, the number of cells with negative volume is considered to be within 0.5% of the total number of cells as a valid

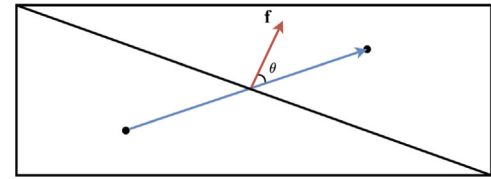


Fig. 5. Cell Non-Orthogonality: This concept refers to the non-orthogonality of cells, where the black dots represent the centroids of the cells, and  $f$  is the normal vector of the shared face. The angle  $\theta$  is defined by the vector normal to the shared face and the line connecting the two centroids.

mesh. Valid meshes are denoted by Yes (Y), and invalid meshes by No (N).

We want to emphasize the distinction between the “Cell Non-Orthogonality” metric and the general concept of mesh orthogonality. The “Cell Non-Orthogonality” metric is primarily introduced to present numerical results in subsequent tables, and it is specifically calculated for 3D cells. However, for structured meshes, the concept of orthogonality applies to both 2D and 3D cases. Mesh orthogonality can be reflected through other metrics, such as the Maximum/Minimum Included Angle or Centroid Skewness.

Thus, mesh orthogonality is not limited to the “Cell Non-Orthogonality” metric alone. In our visualizations of the quality of both 2D and 3D structured meshes, we use the “Minimum Included Angle” metric to visualize mesh quality and orthogonality via heatmaps. This approach allows us to indirectly assess the orthogonality of the mesh, whether it is 2D or 3D.

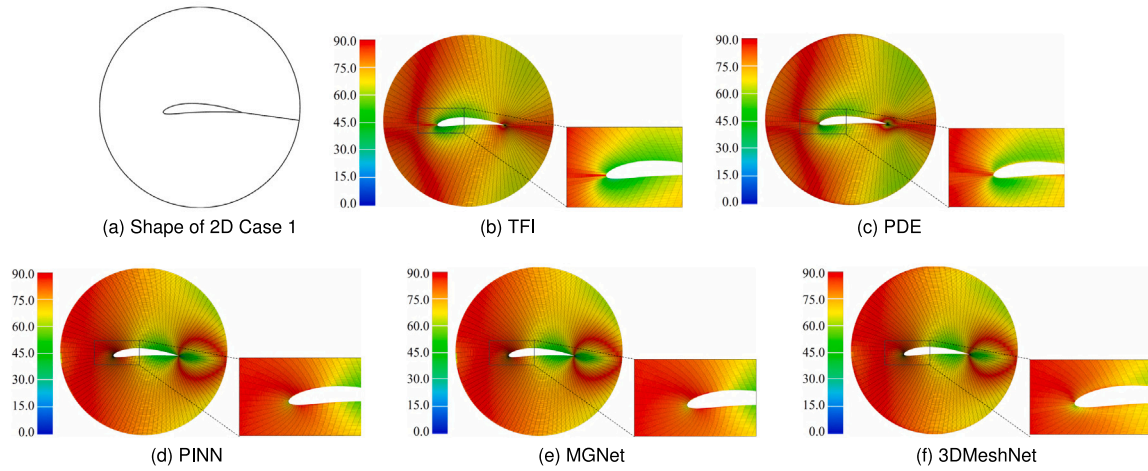
#### 4.1. Experimental results of 3DMeshNet in 2D structured mesh generation

##### 4.1.1. Experimental results of 3DMeshNet in 2D case 1

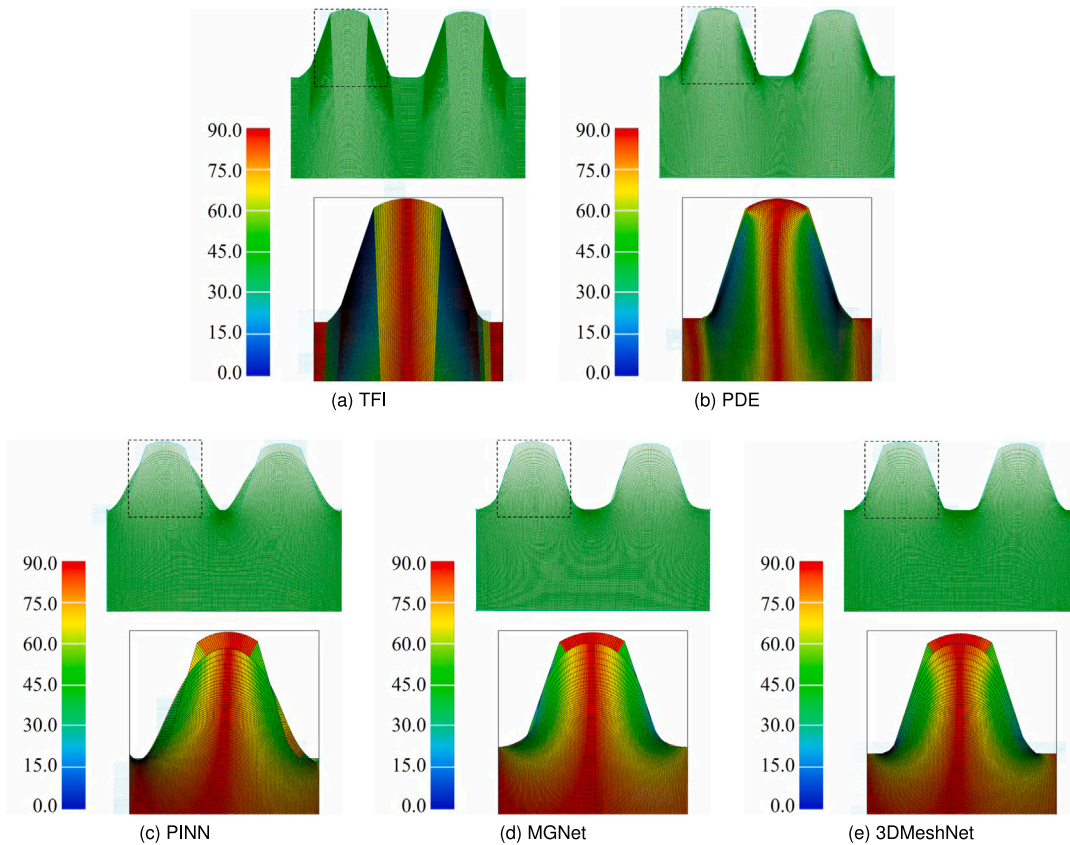
Table 1 and Fig. 6 show the meshing results for 2D Case 1, where 3DMeshNet surpasses PINN and MGNet by achieving a lower loss. It showcases a maximum angle of 103.61 compared to PINN's and MGNet's 104.1 and a minimum angle of 76.31 against PINN's 75.86, indicating improved orthogonality. Compared with TFI and PDE methods, 3DMeshNet yields higher-quality meshes, outperforming PDE's 105.50 and 74.72. Accordingly, 3DMeshNet demonstrates superior orthogonality to both PDE and TFI methods in this test case.

##### 4.1.2. Experimental results of 3DMeshNet in 2D case 2

Table 2 and Fig. 7 illustrate the results of structured mesh generation on 2D Case 2 using TFI, PDE, PINN, MGNet and 3DMeshNet methods. 3DMeshNet's average test set loss 0.0004 exceeds PINN's



**Fig. 6.** Comparison of structured mesh generation results for 2D Case 1 using TFI, PDE, PINN, MGNet, and 3DMeshNet methods. The colors in the subfigures below indicate mesh quality: red represents higher orthogonality, and blue denotes lower.

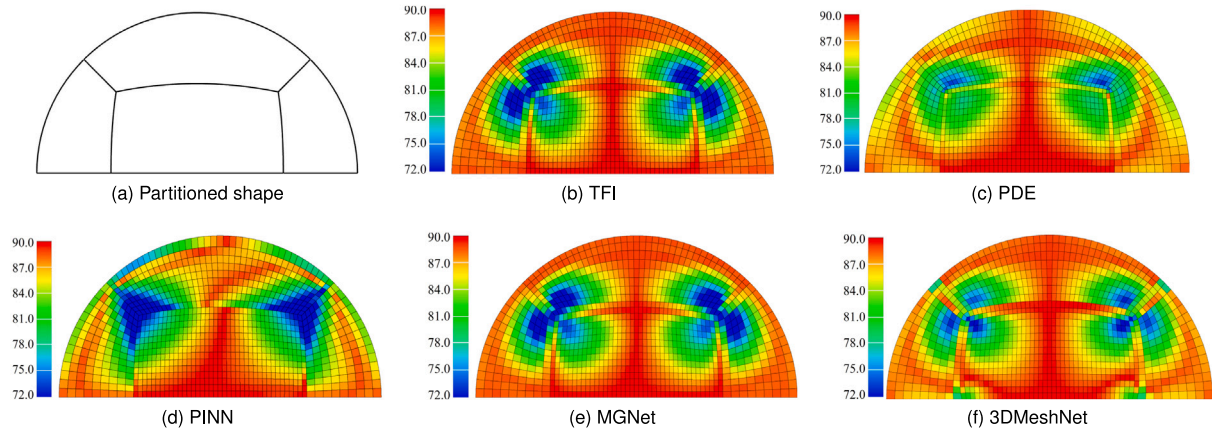


**Fig. 7.** Comparison of structured mesh generation results for 2D Case 2 using TFI, PDE, PINN, MGNet, and 3DMeshNet methods. The colors in the subfigures below indicate mesh quality: red represents higher orthogonality, and blue denotes lower.

0.0040 and MGNet's 0.003. In terms of mesh quality, 3DMeshNet has a Average Maximum Included Angle of 98.32 compared to PINN's 97.87 and a Average Minimum Included Angle of 81.72 compared to PINN's 82.13, and there appears to be no numerical advantage for 3DMeshNet. However, PINN's meshes displayed negative areas and poor boundary fitting capabilities, indicating a failure to learn the shape information accurately, and Mesh Validity is False. Moreover, 3DMeshNet exhibits a lower Skewness Equiangle of 0.09 compared to TFI's 0.31 and PDE's 0.16, indicating a lesser degree of mesh skewness.

#### 4.1.3. Experimental results of 3DMeshNet in 2D case 3

Table 3 illustrates the results of structured mesh generation on the 2D Case 3 using TFI, PDE, PINN, MGNet, and 3DMeshNet. Fig. 8 shows that the mesh quality generated by TFI, PINN, and MGNet around the partitioned corner is unsatisfactory. The mesh quality at the corner points is better for the PDE and 3DMeshNet methods. The mesh generation inside the region is relatively easier, and all four methods generate meshes of good quality within the partitioned shape. Additionally, the shape partition further reduces the difficulty of generating meshes for



**Fig. 8.** Comparison of structured mesh generation results for the 2D Case 3 using TFI, PDE, PINN, MGNet, and 3DMeshNet. Here, we performed a simple manual segmentation of the shape to facilitate better mesh generation. The heatmap colors represent mesh quality: red indicates higher orthogonality, while blue (green) indicates lower orthogonality.

**Table 3**

Comparison of numerical results for structured mesh generation on the 2D Case 3 using TFI, PDE, PINN, MGNet, and 3DMeshNet.

| Method    | Avg. test set loss↓                        | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Mesh validity |
|-----------|--------------------------------------------|------------------------------|------------------------------|---------------------|----------------|---------------|
| TFI       | –                                          | 84.46668                     | 96.25101                     | 0.05442             | 1.34370        | Y             |
| PDE       | –                                          | <b>85.25106</b>              | <b>95.31825</b>              | 0.05667             | 1.33294        | Y             |
| PINN      | $6.61381 \times 10^{-3}$                   | 83.70405                     | 97.00702                     | 0.05620             | 1.34310        | Y             |
| MGNet     | $5.99502 \times 10^{-3}$                   | 84.69762                     | 96.21502                     | 0.05348             | 1.36096        | Y             |
| 3DMeshNet | <b><math>1.30412 \times 10^{-3}</math></b> | 85.19357                     | 95.83388                     | <b>0.05140</b>      | <b>1.33246</b> | Y             |

**Table 4**

Comparison of numerical results for structured mesh generation on the 2D Case 4 using TFI, PDE, PINN, MGNet, and 3DMeshNet.

| Method    | Avg. test set loss↓                        | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Mesh validity |
|-----------|--------------------------------------------|------------------------------|------------------------------|---------------------|----------------|---------------|
| TFI       | –                                          | 85.13000                     | 94.86912                     | 0.05442             | <b>1.79514</b> | Y             |
| PDE       | –                                          | 85.02700                     | 94.92586                     | 0.05667             | 1.79680        | Y             |
| PINN      | $4.65717 \times 10^{-2}$                   | 85.00329                     | 95.01983                     | 0.05620             | 1.79942        | Y             |
| MGNet     | $2.77813 \times 10^{-2}$                   | 85.23825                     | 94.78044                     | 0.05348             | 1.79946        | Y             |
| 3DMeshNet | <b><math>2.43455 \times 10^{-2}</math></b> | <b>85.42071</b>              | <b>94.59833</b>              | <b>0.05140</b>      | 1.80374        | Y             |

complex geometries, enabling 3DMeshNet to generate meshes for a broader range of shapes.

In terms of 2D Case 3 numerical comparison, among the NN-based meshing methods, 3DMeshNet's average test set loss  $1.30412 \times 10^{-3}$  is lower than both PINN and MGNet, and it also outperforms them in metrics such as Average Max. Included Angle, Average Min. Included Angle and Equiangle Skewness. The orthogonality of the generated meshes is slightly lower than that of the PDE method, but the difference is minimal.

#### 4.1.4. Experimental results of 3DMeshNet in 2D case 4

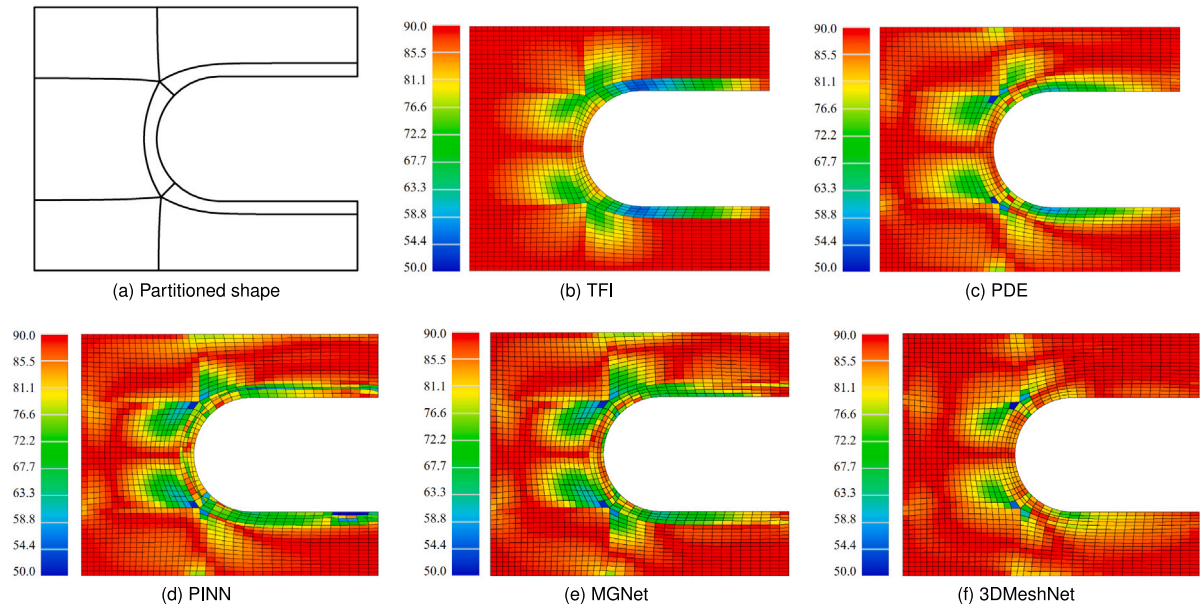
Table 4 and Fig. 9 illustrate the results of structured mesh generation for the 2D Case 4 using TFI, PDE, PINN, MGNet, and 3DMeshNet. Among these methods, 3DMeshNet demonstrates better mesh orthogonality. In the inner region of the U-shape, PINN, and MGNet show some distortion in the mesh lines. PINN, in particular, produces lower-quality meshes along the inner side of the U-shape, which is confirmed by the numerical results. The TFI method generates high-quality meshes for regular, square-like shapes, but some mesh distortion occurs in the inner curved areas. The PDE method generates high-quality meshes for regular shapes and performs well on slightly distorted geometries. For 3DMeshNet, we generate meshes on different partitions of the shape and then merge them. We also present the meshing results of 3DMeshNet on various shapes in Fig. 10 to demonstrate the potential of 3DMeshNet in generating meshes for different shapes.

#### 4.2. Experimental results of 3DMeshNet in 3D structured mesh generation

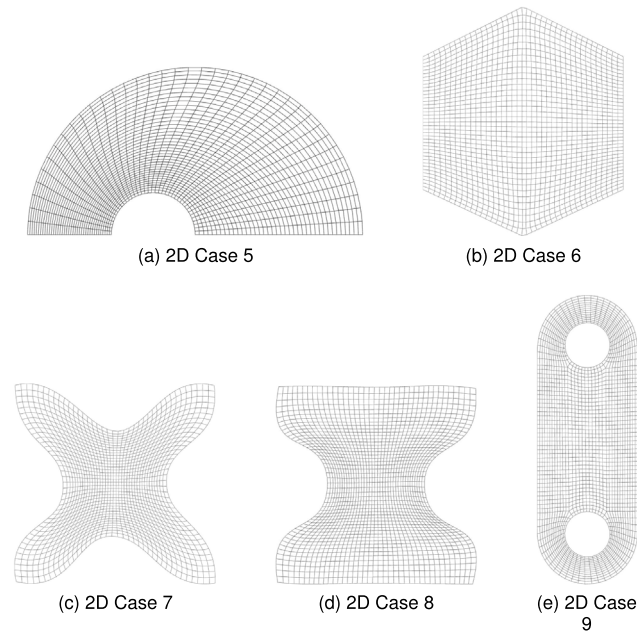
In this section, we perform experiments on four shapes to demonstrate the efficacy of 3DMeshNet in generating 3D structured meshes. To evaluate the quality of the generated meshes, we utilize a range of metrics, including Method, Average Test Set Loss, Average Minimum Included Angle, Average Maximum Included Angle, Skewness Equiangle, Aspect Ratio, Centroid Skewness, Cell Non-Orthogonality and Mesh Validity. Additionally, the Mesh Overhead is used to assess the efficiency of mesh generation.

##### 4.2.1. Experimental results of 3DMeshNet in 3D case 1

Table 5 and Fig. 11 demonstrate that 3DMeshNet achieves the lowest Average Test Set Loss, surpassing PINN and MGNet. Regarding mesh quality, 3DMeshNet achieves the highest Average Minimum Included Angle at 73.19 and the Lowest Average Maximum Included Angle at 107.98, indicating superior orthogonality. It also records the lowest Skewness Equiangle value at 0.19, indicative of a shape nearing the ideal equiangular configuration. A visual analysis of Fig. 11 and its detailed views showcases the significantly inferior mesh quality at the circle's corners with the TFI method relative to 3DMeshNet. In addition, the PINN method demonstrates a limited capability to fit boundaries at corners, while 3DMeshNet exhibits excellent boundary surface adherence, enhancing the overall mesh quality.



**Fig. 9.** Comparison of structured mesh generation results for the 2D Case 4 using TFI, PDE, PINN, MGNet, and 3DMeshNet. Here, we performed a simple manual shape segmentation to facilitate better mesh generation. The heatmap colors represent mesh quality: red indicates higher orthogonality, while blue (green) indicates lower orthogonality.



**Fig. 10.** The mesh generation results of 3DMeshNet on different shapes.

**Table 5**

Numerical results comparison of structured mesh generation on 3D Case 1 using TFI, PDE, PINN, MGNet and 3DMeshNet methods.

| Method    | Avg. test set loss↓                      | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Centroid skewness↓ | Cell non-orthogonality↓ | Mesh validity |
|-----------|------------------------------------------|------------------------------|------------------------------|---------------------|----------------|--------------------|-------------------------|---------------|
| TFI       | –                                        | 71.29516                     | 109.84139                    | 0.22069             | <b>1.31980</b> | 0.11211            | 21.63536                | Y             |
| PDE       | –                                        | 73.14630                     | 108.09262                    | 0.20103             | 1.37678        | 0.07211            | 18.04514                | Y             |
| PINN      | $7.767 \times 10^{-4}$                   | 69.93509                     | 109.81369                    | 0.22440             | 1.39929        | 0.09855            | 22.00432                | N             |
| MGNet     | $5.652 \times 10^{-4}$                   | 73.03043                     | 108.03066                    | 0.20105             | 1.37507        | <b>0.07003</b>     | 18.11415                | Y             |
| 3DMeshNet | <b><math>1.205 \times 10^{-4}</math></b> | <b>73.19016</b>              | <b>107.98604</b>             | <b>0.19992</b>      | 1.37739        | 0.07106            | <b>18.02890</b>         | Y             |

#### 4.2.2. Experimental results of 3DMeshNet in 3D case 2

Table 6 and Fig. 12 show 3DMeshNet stands out with the lowest Test Set Loss at  $1.33\text{e}-05$ , better than PINN's  $9.32\text{e}-05$ . In terms of angular metrics, 3DMeshNet achieves superior Minimum and Average

Maximum Included Angles at 75.11 and 104.90, respectively, illustrating its optimal angular conformity. The Aspect Ratio of 1.59 for 3DMeshNet suggests a closer approximation to the ideal than PINN's 1.60. Furthermore, 3DMeshNet excels in metrics of Centroid Skewness

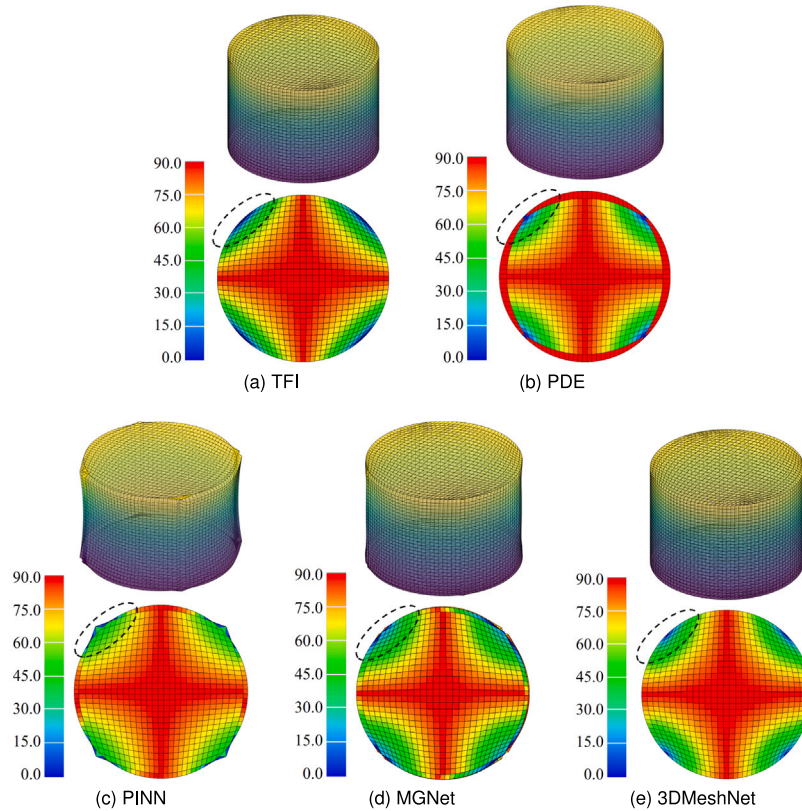


Fig. 11. Comparison of structured mesh generation results for 3D Case 1 employing TFI, PDE, PINN, MGNet, and 3DMeshNet methods. The subfigures below the 3D graph are visualizations of the 3D block's cross-sections, where colors indicate mesh quality: red for higher orthogonality and blue for lower.

Table 6

Numerical results comparison of structured mesh generation on 3D Case 2 using TFI, PDE, PINN, MGNet and 3DMeshNet methods.

| Method    | Avg. test set loss↓      | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓ | Centroid skewness↓ | Cell non-orthogonality↓ | Mesh validity |
|-----------|--------------------------|------------------------------|------------------------------|---------------------|---------------|--------------------|-------------------------|---------------|
| TFI       | –                        | 72.02420                     | 107.98571                    | 0.20151             | 1.61927       | 0.05835            | 18.30840                | Y             |
| PDE       | –                        | 73.91890                     | 106.08204                    | 0.17957             | 1.59924       | 0.05029            | 16.24168                | Y             |
| PINN      | $9.32628 \times 10^{-5}$ | 74.10544                     | 105.89219                    | 0.17752             | 1.60238       | 0.04914            | 16.03306                | Y             |
| MGNet     | $6.45489 \times 10^{-5}$ | 74.34177                     | 105.66810                    | 0.17471             | 1.59769       | 0.04785            | 15.81563                | Y             |
| 3DMeshNet | $1.33345 \times 10^{-5}$ | 75.11612                     | 104.90331                    | 0.16605             | 1.59037       | 0.04526            | 15.06806                | Y             |

Table 7

Numerical results comparison of structured mesh generation on 3D Case 3 using TFI, PDE, PINN, MGNet and 3DMeshNet methods.

| Method    | Avg. test set loss↓      | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓ | Centroid skewness↓ | Cell non-orthogonality↓ | Mesh validity |
|-----------|--------------------------|------------------------------|------------------------------|---------------------|---------------|--------------------|-------------------------|---------------|
| TFI       | –                        | 67.52675                     | 112.47325                    | 0.24970             | 2.22038       | 0.09925            | 25.20217                | Y             |
| PDE       | –                        | 73.32343                     | 107.01437                    | 0.19070             | 2.31726       | 0.05531            | 18.35843                | Y             |
| PINN      | $2.76692 \times 10^{-4}$ | 61.91867                     | 117.90227                    | 0.31438             | 3.10559       | 0.16059            | 31.22226                | N             |
| MGNet     | $9.34615 \times 10^{-5}$ | 74.66378                     | 105.52406                    | 0.17388             | 2.30234       | 0.04624            | 16.49563                | Y             |
| 3DMeshNet | $1.65482 \times 10^{-5}$ | 74.92872                     | 105.14837                    | 0.16992             | 2.29716       | 0.04418            | 16.15760                | Y             |

and Cell Non-Orthogonality, with the lowest observed values at 0.04 and 15.06, respectively, indicating a higher fidelity of mesh quality in comparison to PINN, MGNet, TFI, and PDE.

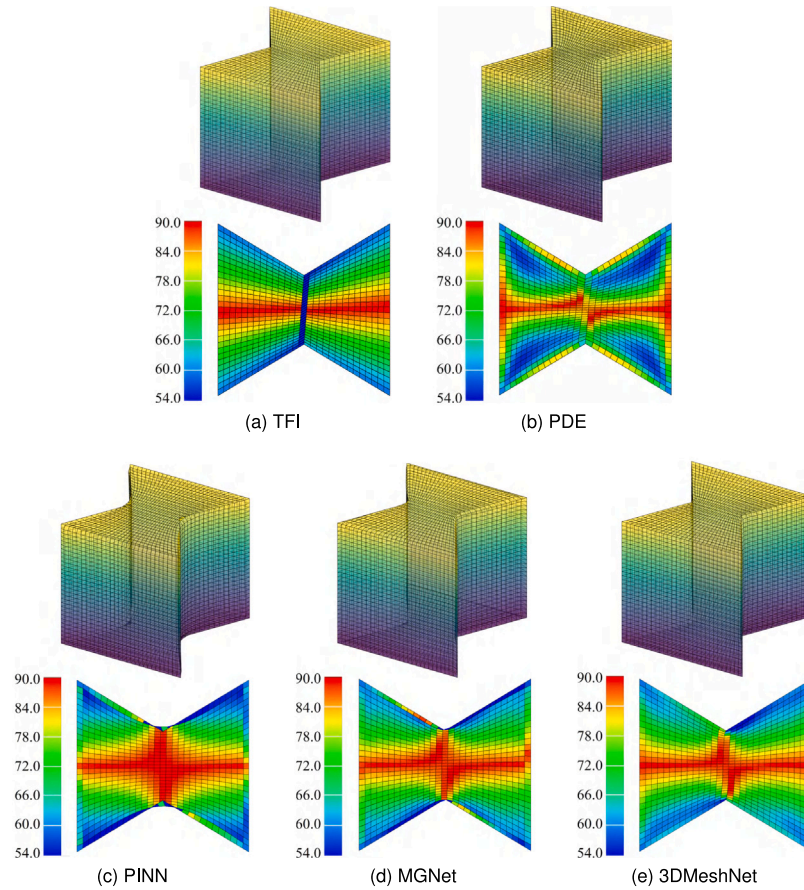
#### 4.2.3. Experimental results of 3DMeshNet in 3D case 3

Table 7 and Fig. 13 demonstrate that 3DMeshNet outperforms all others in Centroid Skewness and Cell Non-Orthogonality, achieving the lowest values of 0.044 and 16.15, respectively, indicating superior mesh quality. Moreover, Fig. 13 shows that 3DMeshNet enhances mesh orthogonality in the highlighted region compared to PDE and TFI methods, while the PINN method exhibits significant mesh cell

distortion. MGNet performs better than PINN but is still slightly inferior to 3DMeshNet.

#### 4.2.4. Experimental results of 3DMeshNet in 3D case 4

Table 8 and Fig. 14 present the Average Test Set Loss with 3DMeshNet, achieving a value of  $2.26 \times 10^{-5}$ , representing an order of magnitude improvement over PINN's  $6.38 \times 10^{-4}$ . Furthermore, 3DMeshNet outperforms TFI, PDE, and MGNet by achieving average minimum and average maximum included angles of 77.24 and 102.81, respectively. In contrast, the mesh generated by PINN is invalid. 3DMeshNet accurately captures the object's surface shape while maintaining favorable



**Fig. 12.** Comparison of structured mesh generation results on 3D Case 2 using TFI, PDE, PINN, MGNet, and 3DMeshNet methods. The subfigures below the 3D graph are visualizations of the 3D block's cross-sections, where colors indicate mesh quality: red for higher orthogonality and blue for lower.

**Table 8**

Numerical results comparison of structured mesh generation on 3D Case 4 using TFI, PDE, PINN, MGNet and 3DMeshNet methods.

| Method    | Avg. test set loss↓                        | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Centroid skewness↓ | Cell non-orthogonality↓ | Mesh validity |
|-----------|--------------------------------------------|------------------------------|------------------------------|---------------------|----------------|--------------------|-------------------------|---------------|
| TFI       | —                                          | 72.60045                     | 107.57008                    | 0.19559             | 2.92629        | 0.09432            | 17.96599                | Y             |
| PDE       | —                                          | 75.68917                     | 104.43286                    | 0.16128             | 2.87590        | 0.06029            | 14.77475                | Y             |
| PINN      | $6.38450 \times 10^{-4}$                   | <b>78.82539</b>              | <b>101.18939</b>             | <b>0.12520</b>      | <b>2.75698</b> | <b>0.02872</b>     | <b>11.47250</b>         | N             |
| MGNet     | $1.64892 \times 10^{-4}$                   | 76.76285                     | 103.36702                    | 0.14917             | 2.86600        | 0.04316            | 13.51371                | N             |
| 3DMeshNet | <b><math>2.26415 \times 10^{-5}</math></b> | 77.24883                     | 102.81078                    | 0.14345             | 2.82172        | 0.03892            | 13.12343                | Y             |

included angles, resulting in higher quality mesh generation than PINN, as illustrated in Fig. 14.

#### 4.2.5. Experimental results of 3DMeshNet in 3D case 5

Table 9 and Fig. 15 show that 3DMeshNet generates orthogonal structured meshes with better quality on the 3D Case 5. Among NN-based methods, PINN struggles to generate good meshes around curved and corner regions, while the meshes generated by MGNet deviate from the geometry boundaries. In contrast, 3DMeshNet produces more uniform meshes than PINN and MGNet. Additionally, 3DMeshNet outperforms TFI and PDE methods regarding mesh quality, as indicated by the smaller green range in the heatmap.

#### 4.2.6. Experimental results of 3DMeshNet in 3D case 6

Table 10 and Fig. 16 presents a comparison of the structured mesh generation results for five methods on the 3D Case 6, which represents a 3D airfoil. The mesh generated by the TFI method shows poor mesh quality along the airfoil edges. Compared to TFI, the PDE method improves mesh orthogonality. PINN produces poor-quality meshes at the airfoil's leading edge, with noticeable mesh distortion. MGNet

shows some improvement in mesh orthogonality. The meshes generated by 3DMeshNet exhibit good quality both at the leading edge and along the edges of the airfoil.

#### 4.2.7. Experimental results of 3DMeshNet in 3D case 7

Table 11 and Fig. 17 present a comparison of structured mesh generation results for five methods applied to the 3D Case 7, which represents a 3D twisted airfoil commonly used in propellers. The mesh generated by the TFI method shows poor mesh quality along the edges of the airfoil. Compared to TFI, the PDE method improves mesh orthogonality. PINN generates meshes with noticeable distortions at the leading edge, where the mesh quality is poor and exhibits a concave shape at the leading edge. MGNet improves upon this by reducing the concave shape at the twisted airfoil's leading edge and increasing mesh orthogonality. The meshes generated by 3DMeshNet exhibit good quality at the leading edge and along the edges, without any concave deformation. It is worth noting that, for visualization purposes, we used relatively small mesh sizes; however, in practical applications, larger mesh sizes are required, which can further improve the quality of the generated meshes.

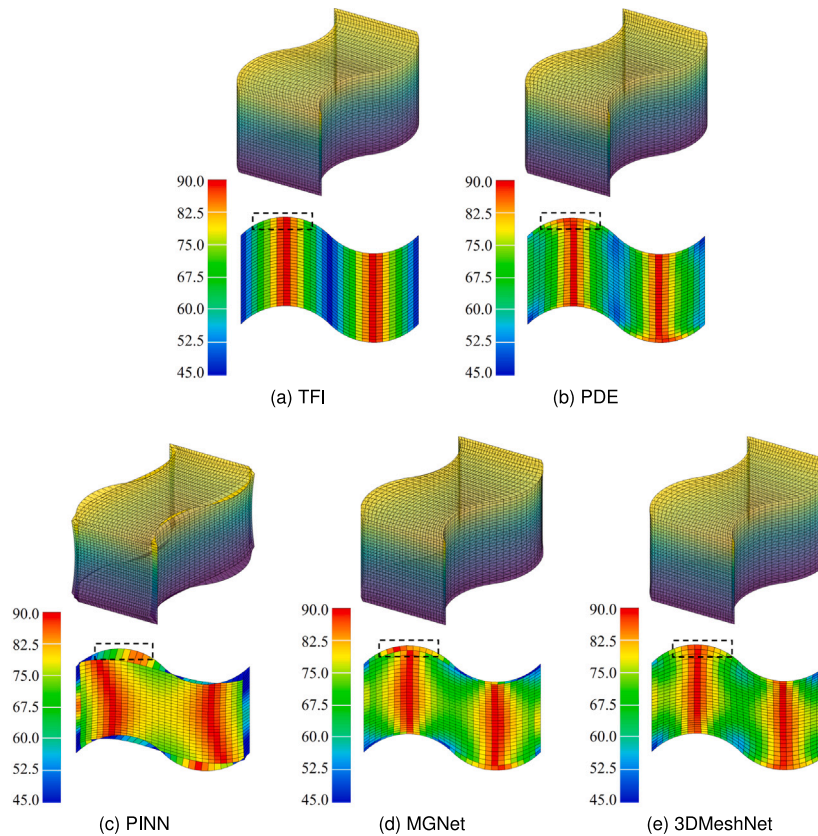


Fig. 13. Comparison of structured mesh generation results on 3D Case 3 using TFI, PDE, PINN, MGNet, and 3DMeshNet methods. The subfigures below the 3D graph are visualizations of the 3D block's cross-sections, where colors indicate mesh quality: red for higher orthogonality and blue for lower.

Table 9

Comparison of numerical results for structured mesh generation on the 3D Case 5 using TFI, PDE, PINN, MGNet, and 3DMeshNet methods.

| Method    | Avg. test set loss                         | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Centroid skewness↓ | Cell non-orthogonality↓ | Mesh validity |
|-----------|--------------------------------------------|------------------------------|------------------------------|---------------------|----------------|--------------------|-------------------------|---------------|
| TFI       | –                                          | 74.24898                     | 106.24238                    | 0.18063             | 4.41722        | 0.05383            | 15.99174                | Y             |
| PDE       | –                                          | 74.26362                     | 106.22297                    | 0.18045             | 4.40766        | <b>0.05380</b>     | 16.06382                | Y             |
| PINN      | $2.04976 \times 10^{-2}$                   | 68.99052                     | 111.50835                    | 0.23977             | 4.31157        | 0.10103            | 22.63090                | N             |
| MGNet     | $6.64342 \times 10^{-3}$                   | 70.87309                     | 109.60732                    | 0.21804             | <b>4.25107</b> | 0.07897            | 19.54598                | Y             |
| 3DMeshNet | <b><math>1.62344 \times 10^{-3}</math></b> | <b>74.48923</b>              | <b>106.00562</b>             | <b>0.17784</b>      | 4.42735        | 0.05408            | <b>15.66773</b>         | Y             |

Table 10

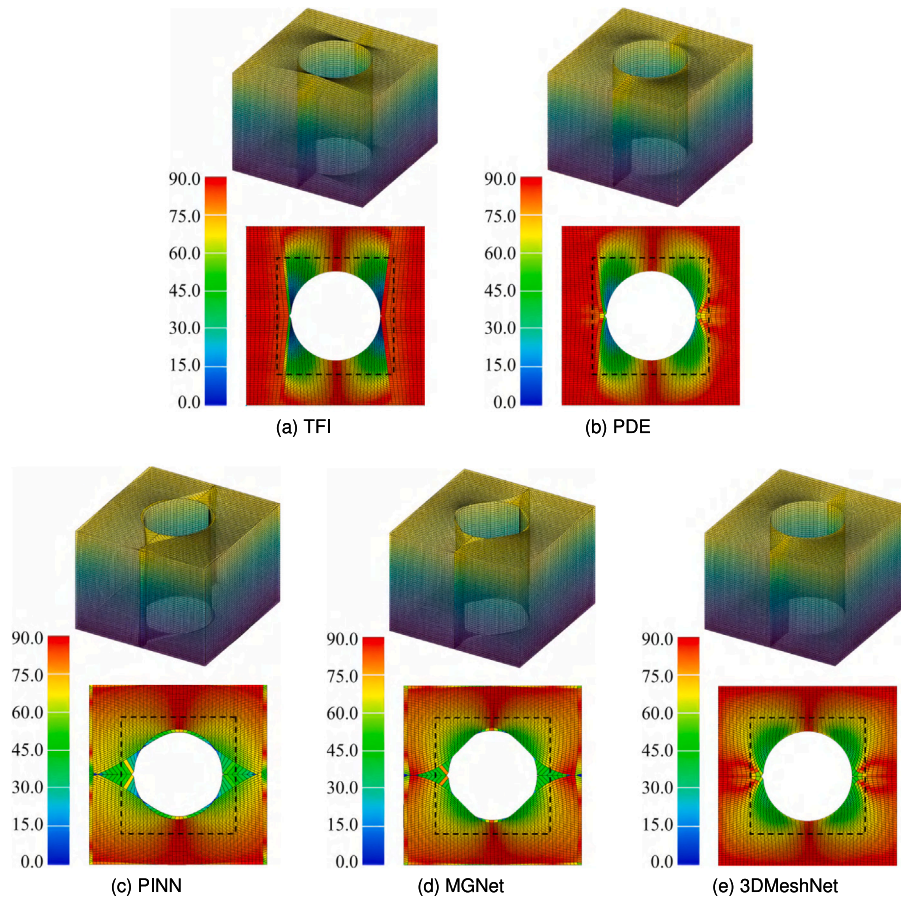
Numerical results comparison of structured mesh generation on 3D Case 6 using TFI, PDE, PINN, MGNet and 3DMeshNet methods.

| Method    | Avg. test set loss↓                        | Average Min. included angle↑ | Average Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓   | Centroid skewness↓ | Cell non-orthogonality↓ | Mesh validity |
|-----------|--------------------------------------------|------------------------------|------------------------------|---------------------|-----------------|--------------------|-------------------------|---------------|
| TFI       | –                                          | 59.11139                     | 121.02309                    | 0.34591             | 17.29985        | 0.20078            | 34.58178                | Y             |
| PDE       | –                                          | 64.08320                     | 115.06506                    | 0.29305             | 18.11925        | 0.14542            | 29.38133                | Y             |
| PINN      | $4.02049 \times 10^{-3}$                   | 58.18378                     | 122.05775                    | 0.35726             | <b>15.07492</b> | 0.19735            | 35.15377                | N             |
| MGNet     | $1.06083 \times 10^{-3}$                   | 62.93184                     | 117.28690                    | 0.30403             | 16.83074        | 0.14855            | 30.14761                | Y             |
| 3DMeshNet | <b><math>8.70566 \times 10^{-4}</math></b> | <b>65.85486</b>              | <b>114.29993</b>             | <b>0.27020</b>      | 18.97747        | <b>0.11095</b>     | <b>25.75852</b>         | Y             |

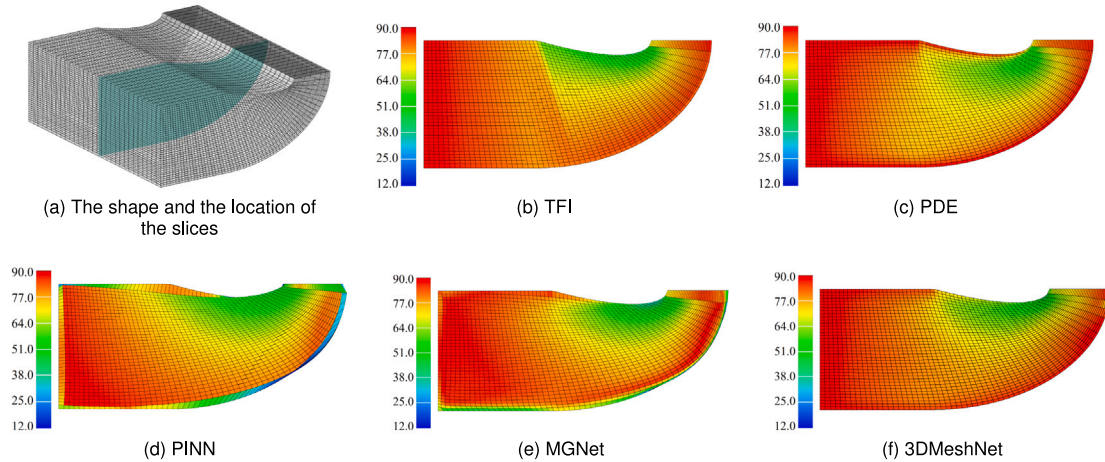
Table 11

Numerical results comparison of structured mesh generation on 3D Case 7 using TFI, PDE, PINN, MGNet, and 3DMeshNet methods.

| Method    | Avg. test set loss↓                        | Average Min. included angle↑ | Max. included angle↓ | Equiangle skewness↓ | Aspect ratio↓  | Centroid skewness↓ | Cell non-orthogonality↓ | Mesh validity |
|-----------|--------------------------------------------|------------------------------|----------------------|---------------------|----------------|--------------------|-------------------------|---------------|
| TFI       | –                                          | 44.62920                     | 135.34305            | 0.50464             | <b>9.89881</b> | 0.35034            | 46.44378                | Y             |
| PDE       | –                                          | 56.30231                     | 123.67940            | 0.37503             | 11.44454       | 0.22022            | 34.65019                | Y             |
| PINN      | $1.06711 \times 10^{-4}$                   | 45.57033                     | 134.54420            | 0.49721             | 11.09301       | 0.34978            | 46.46335                | N             |
| MGNet     | $9.96521 \times 10^{-5}$                   | 50.50663                     | 129.54531            | 0.44171             | 10.89372       | 0.29631            | 41.45486                | N             |
| 3DMeshNet | <b><math>5.88672 \times 10^{-5}</math></b> | <b>59.23171</b>              | <b>120.82809</b>     | <b>0.34303</b>      | 11.39046       | <b>0.19541</b>     | <b>32.59744</b>         | Y             |



**Fig. 14.** Comparison of structured mesh generation results on 3D Case 4 using TFI, PDE, PINN, MGNet, and 3DMeshNet methods. The subfigures below the 3D graph are visualizations of the 3D block's cross-sections, where colors indicate mesh quality: red for higher orthogonality and blue (green) for lower.



**Fig. 15.** Comparison of structured mesh generation results for the 3D Case 5 using TFI, PDE, PINN, MGNet, and 3DMeshNet. We present the mesh slices of this geometry, shown in (a), and use heatmaps in panels (b) to (f) to display the mesh quality. The heatmap colors represent mesh quality: red indicates higher orthogonality, while blue (green) indicates lower orthogonality.

#### 4.3. Meshing overhead

Here, we compare 3DMeshNet's meshing overhead with TFI, PDE, PINN, MGNet, and 3DMeshNet methods to validate 3DMeshNet's applicability in engineering applications as a reliable mesh generator. We test different mesh sizes for 2D, including  $30^2$  and  $282^2$ , and for 3D, including  $30^3$  and  $100^3$ , to evaluate the impact of mesh density on the mesh generation systems. In Table 12, we present the performance of

the various methods based on execution times. All meshing processes are single-threaded in Python on the Intel Core i5-1240P CPU, and each meshing operation is repeated ten times. The average time is recorded as the final overhead.

Generally, meshing overhead increases as mesh sizes grow larger. TFI exhibits linear overhead scaling with mesh size. In 2D cases, a 7.9x increase in mesh scale leads to an approximately 8.5x rise in

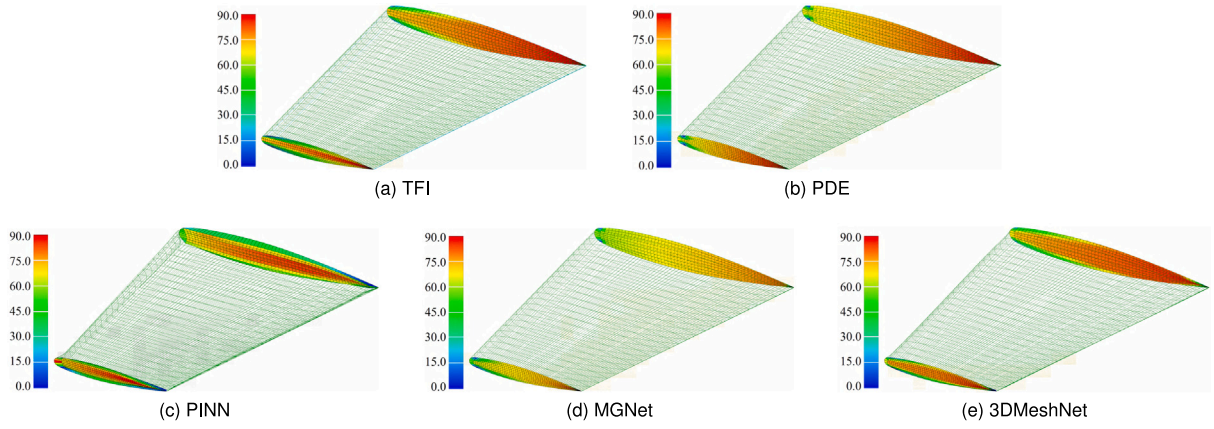


Fig. 16. Comparison of structured mesh generation results for the 3D Case 6 using TFI, PDE, PINN, MGNet, and 3DMeshNet. Here, we present a visualization of the mesh slices for the front and rear surfaces of the wing. The heatmap colors represent mesh quality: red indicates higher orthogonality, while blue (green) indicates lower orthogonality.

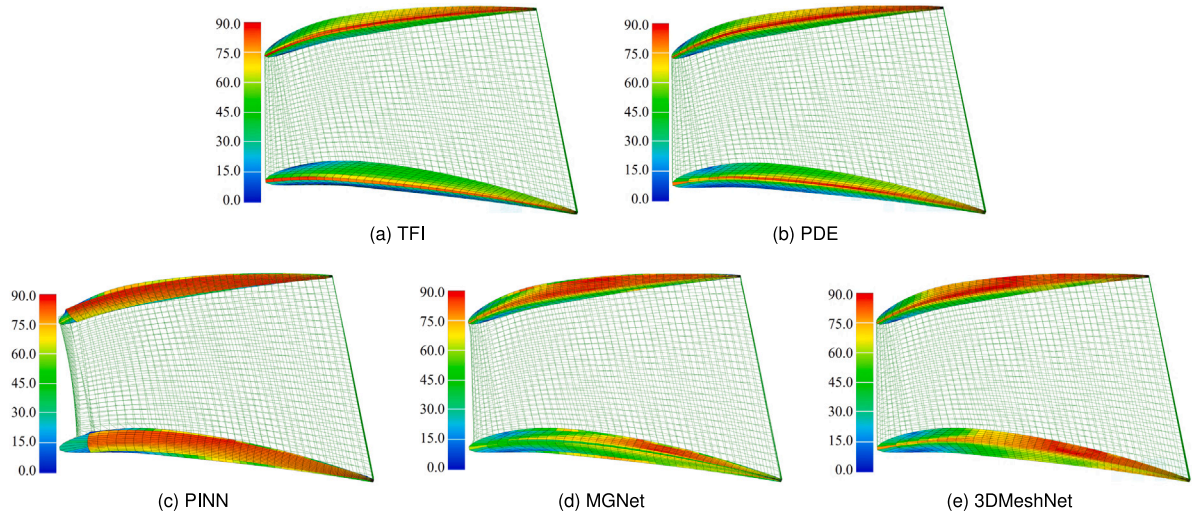


Fig. 17. Comparison of structured mesh generation results for the 3D Case 7 using TFI, PDE, PINN, MGNet, and 3DMeshNet. Here, we present a visualization of the mesh slices for the front and rear surfaces of the twisted wing. The heatmap colors represent mesh quality: red indicates higher orthogonality, while blue (green) indicates lower orthogonality.

Table 12

Meshing overhead results in 2D and 3D datasets. The mesh size of 2D is  $30^2$  and  $282^2$ ; 3D is  $30^3$  and  $100^3$ .

| Method    | $30^2$         | $282^2$        | $30^3$         | $100^3$        |
|-----------|----------------|----------------|----------------|----------------|
| TFI       | 0.58299        | 1.17128        | 0.43193        | 14.67969       |
| PDE       | 187.83363      | 1623.18041     | 828.05571      | 27328.36674    |
| PINN      | 0.00449        | 0.00349        | 0.07862        | 2.38957        |
| MGNet     | <b>0.00399</b> | 0.00449        | <b>0.07583</b> | 2.41105        |
| 3DMeshNet | 0.00499        | <b>0.00346</b> | 0.08072        | <b>2.27391</b> |

generation time. In 3D scenarios, a 37x increase in mesh scale corresponds to a 34x increase in generation time. However, the algebraic interpolation method frequently yields skewed or distorted meshes along complex boundary curves, potentially hindering convergence or causing numerical issues during numerical analysis.

Regarding the PDE method, overhead computation involves a finite number of elliptic smoothing iterations. Results indicate that the PDE method needs more computational time, primarily due to the expensive numerical iterations of the elliptic PDE system. It takes less than 27 min for a mesh size of  $300^2$  and approximately seven hours for the largest size of  $100^3$ . The PDE method's execution time significantly surpasses that of algebraic and neural network-based methods.

Table 12 suggests that 3DMeshNet effectively balances mesh quality and meshing overhead, generating high-quality meshes at a lower cost

than the PDE method. Its near-linear scaling of overhead and independent feed-forward prediction make parallelization easy on modern systems, leading to faster execution times. Furthermore, compared to PINN and MGNet, 3DMeshNet reduces training time by 85%. Consequently, 3DMeshNet demonstrates outstanding performance in both training time and meshing overhead.

#### 4.4. Ablation experiments

This section presents ablation experiments evaluating mesh generation on 2D and 3D datasets. This section evaluates the mesh generation capabilities on four different models: 3DMeshNet without GP (Gradient Projection), which excludes the Gradient Projection module (described in Section 3.5); 3DMeshNet without Reweight, which removes the loss function reweighting module (described in Section 3.4); and 3DMeshNet without FD, which omits the finite difference module (described in Section 3.3). Lastly, 3DMeshNet refers to the model proposed in this paper.

As depicted in Table 13, the numerical results for both 2D and 3D datasets show that 3DMeshNet without FD and 3DMeshNet achieve the best performance, while 3DMeshNet without Reweight produces the worst results. This demonstrates that the loss function reweighting has the most significant impact on the model's numerical performance. It is crucial to appropriately balance the loss of governing differential

**Table 13**

The ablation experiments on various 2D and 3D datasets evaluate the metrics of maximum and minimum angles. The optimal mesh quality is highlighted in bold.

| Dataset                    | 2D                                   |                                       | 3D                                   |                                       |                                       |                                |
|----------------------------|--------------------------------------|---------------------------------------|--------------------------------------|---------------------------------------|---------------------------------------|--------------------------------|
|                            | 2D Case 10                           | 2D Case 11                            | 3D Case 8                            | 3D Case 9                             | 3D Case 10                            | 3D Case 11                     |
| 3DMeshNet without GP       | 81.70413/<br>99.14233                | 64.12264/<br>116.19650                | 83.05948<br>96.46830                 | 79.66622/<br>100.36920                | 74.50760<br>106.65151                 | 68.61361/<br>112.97824         |
| 3DMeshNet without Reweight | 78.241011/<br>102.16700              | 60.82981/<br>119.47654                | 74.29541/<br>106.19347               | 77.49883/<br>102.37844                | 71.36079/<br>109.39217                | 58.80061/<br>122.19004         |
| 3DMeshNet without FD       | <b>82.54338</b> /<br><b>98.19871</b> | 64.01433/<br>116.32903                | 85.49096/<br>93.96115                | <b>79.98722</b> /<br><b>100.05296</b> | 74.04181/<br>106.19268                | <b>68.65815</b> /<br>112.98135 |
| 3DMeshNet                  | 82.21093/<br>98.57325                | <b>65.36629</b> /<br><b>115.00787</b> | <b>85.93510</b> /<br><b>93.52956</b> | 78.76921/<br>101.24903                | <b>75.50764</b> /<br><b>105.58628</b> | 68.649336/<br><b>112.95190</b> |

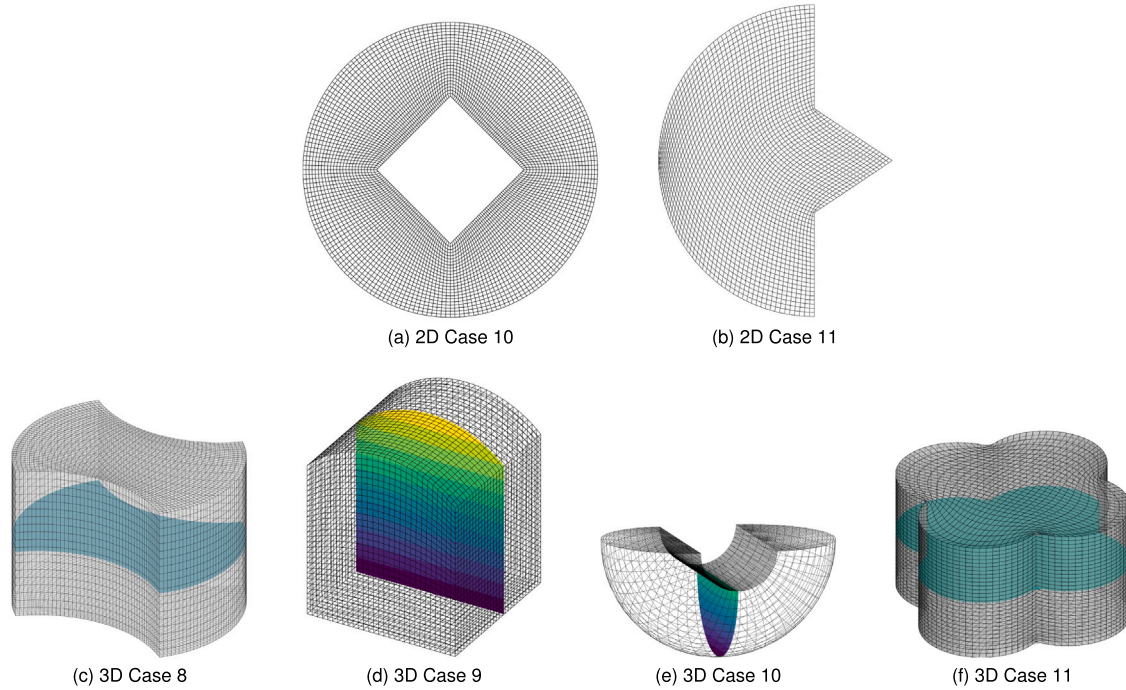


Fig. 18. The meshing results for the different shapes used in the ablation experiments with 3DMeshNet.

equations and the loss of geometric surface constraint during training. The numerical results of 3DMeshNet without FD and 3DMeshNet are similar, but 3DMeshNet requires less training overhead. Compared to 3DMeshNet without FD, training can be completed in less time. 3DMeshNet achieves a balance between training time and accuracy. For the 2D and 3D datasets presented in the ablation experiments, the meshes generated using the 3DMeshNet method are shown in Fig. 18.

Fig. 19 depicts the loss values on the test set for 3DMeshNet without GP, 3DMeshNet without Reweight, 3DMeshNet without FD, and 3DMeshNet. The convergence trends of 3DMeshNet without GP, 3DMeshNet without FD, and 3DMeshNet are similar, but 3DMeshNet achieves the lowest final loss. The loss during the convergence process of 3DMeshNet without GP is less smooth, with larger fluctuations. The convergence of 3DMeshNet without FD is similar to 3DMeshNet without GP. On the other hand, 3DMeshNet without loss function reweighting shows the poorest convergence.

## 5. Conclusion

While deep learning has been applied to various aspects of numerical analysis, its utilization in structured mesh generation is an emerging field with considerable potential. We introduce 3DMeshNet, a neural network-based differential technique for creating 3D structured meshes. 3DMeshNet takes surface points on geometric shapes

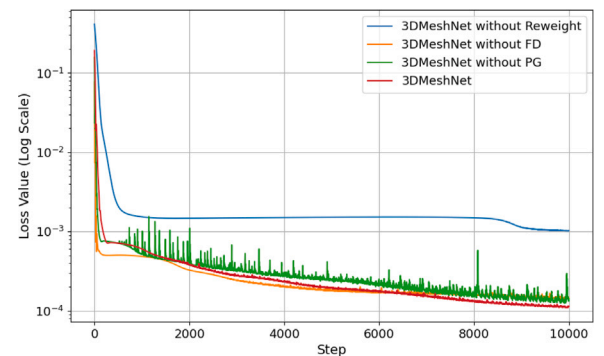


Fig. 19. Comparison of average test set loss on 3DMeshNet without GP, 3DMeshNet without Reweight, 3DMeshNet without FD and 3DMeshNet.

as input to learn the potential mapping between parametric and computational spaces via a neural network. It outputs a 3D structured mesh with a user-defined number of cells to partition the domain. The network approximates the mapping solution by minimizing the weighted residuals of the governing equation and surface constraint

terms. Mesh point calculation is executed through feedforward prediction using deep neural networks, eliminating the substantial overhead of solving PDEs. By incorporating loss function reweighting, gradient projection, and FD methods, we fasten the convergence of the network and accelerate training. Experiments show that 3DMeshNet is quick and robust, surpassing current advanced neural network-based generators and producing quality meshes equal to or greater than traditional methods. 3DMeshNet significantly reduces training time by up to 85% compared to other neural network-based methods while achieving mesh overheads that are 4 to 8 times lower than those produced by the TFI method.

However, 3DMeshNet still has some limitations. Since neural networks are continuous and differentiable, their performance on discontinuous functions, such as sharp corners, is limited. Therefore, the proposed method is best suited for mesh generation in common engineering structures like airfoils, pipelines, and turbine blades. To improve its generality, domain decomposition could be applied to avoid directly generating meshes in sharp regions.

Additionally, the convergence efficiency of the 3DMeshNet network still needs improvement. For instance, when meshing with similar shapes, we could leverage transfer learning or meta-learning techniques to enhance the network's convergence efficiency. In cases where the generated meshes have poor quality at large scales or high resolutions, we could use the meshes produced by 3DMeshNet as initial meshes and then apply mesh smoothing techniques as a post-processing step to improve the mesh quality. Alternatively, we could first generate smaller-scale meshes. The mesh points from these smaller meshes could then be used as additional observation points for further training. This would provide supplementary prior mesh information to help improve the generation quality of large-scale meshes.

In our future work, we will focus on enhancing the learning ability and generalization of 3DMeshNet to generate high-quality 3D structured meshes for more complex shapes. Additionally, we are exploring the integration of the NN-based meshing method with domain decomposition techniques to improve the practicality of the meshing process.

### CRedit authorship contribution statement

**Jiaming Peng:** Writing – original draft. **Xinhai Chen:** Writing – review & editing. **Jie Liu:** Funding acquisition.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgments

This research was partially supported by the Natural Science Foundation of Hunan Province, China (2024JJ6468), the National Natural Science Foundation of China (12402349), the Youth Foundation of the National University of Defense Technology (ZK2023-11), and the National Key Research and Development Program of China (2021YFB0300101).

### Data availability

Data will be made available on request.

### References

- [1] John R. Chawner, Nigel J. Taylor, Progress in geometry modeling and mesh generation toward the CFD vision 2030, in: AIAA Aviation 2019 Forum, 2019, p. 2945.
- [2] Toshiyuki Hayase, Numerical simulation of real-world flows, *Fluid Dyn. Res.* 47 (5) (2015) 051201.
- [3] Xinhai Chen, Jie Liu, Chunye Gong, Shengguo Li, Yufei Pang, Bo Chen, MVE-Net: An automatic 3-D structured mesh validity evaluation framework using deep neural networks, *Computer-Aided Des.* 141 (2021) 103104.
- [4] Jeffrey P Slotnick, Abdollah Khodadoust, Juan Alonso, David Darmofal, William Gropp, Elizabeth Lurie, Dimitri J Mavriplis, *CFD Vision 2030 Study: a Path to Revolutionary Computational Aerosciences*, Technical Report, 2014.
- [5] Steven J. Owen, Tim R. Shelton, Evaluation of grid-based hex meshes for solid mechanics, *Eng. Comput.* 31 (2015) 529–543.
- [6] Anders Logg, Efficient representation of computational meshes, *Int. J. Comput. Sci. Eng.* 4 (4) (2009) 283–295.
- [7] Xinhai Chen, Jie Liu, Junjun Yan, Zhichao Wang, Chunye Gong, An improved structured mesh generation method based on physics-informed neural networks, 2022, arXiv preprint [arXiv:2210.09546](https://arxiv.org/abs/2210.09546).
- [8] Xinhai Chen, Tiejun Li, Qian Wan, Xiaoyu He, Chunye Gong, Yufei Pang, Jie Liu, MGNNet: a novel differential mesh generation method based on unsupervised neural networks, *Eng. Comput.* 38 (5) (2022) 4409–4421.
- [9] C.B. Allen, Towards automatic structured multiblock mesh generation using improved transfinite interpolation, *Internat. J. Numer. Methods Engrg.* 74 (5) (2008) 697–733.
- [10] Michael Turner, *High-Order Mesh Generation for CFD Solvers* (Ph.D. thesis), Imperial College London, 2017.
- [11] Octavi Obiols-Sales, Abhinav Vishnu, Nicholas Malaya, Aparna Chandramowlishwaran, CFDNet: A deep learning-based accelerator for fluid simulations, in: *Proceedings of the 34th ACM International Conference on Supercomputing*, 2020, pp. 1–12.
- [12] Lu Lu, Pengzhan Jin, George Em Karniadakis, DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators, 2019, arXiv preprint [arXiv:1910.03193](https://arxiv.org/abs/1910.03193).
- [13] H. Jilani, Ardeshir Bahreininejad, M.T. Ahmadi, Adaptive finite element mesh triangulation using self-organizing neural networks, *Adv. Eng. Softw.* 40 (11) (2009) 1097–1103.
- [14] Zengsheng Liu, Shizhao Chen, Xiang Gao, Xiang Zhang, Chunye Gong, Chuanfu Xu, Jie Liu, Isplitter: an intelligent and automatic surface mesh generator using neural networks and splitting lines, *Adv. Aerodyn.* 5 (1) (2023) 1–25.
- [15] Sumedh Soman, Ninad Mehendale, Faster and efficient tetrahedral mesh generation using generator neural networks for 2D and 3D geometries, *Neural Comput. Appl.* (2023) 1–9.
- [16] Andreas Lintermann, Computational meshing for CFD simulations, *Clin. Biomed. Eng. Hum. Nose: A Comput. Fluid Dyn. Approach* (2021) 85–115.
- [17] Josep Sarate Ramos, Eloi Ruiz-Gironés, Francisco Javier Roca Navarro, Unstructured and semi-structured hexahedral mesh generation methods, *Comput. Technol. Rev.* 10 (2014) 35–64.
- [18] Marshall W. Bern, Paul E. Plassmann, Mesh generation., *Handb. Comput. Geom.* 38 (2000).
- [19] Ivo Babuska, Modeling, Mesh Generation, and Adaptive Numerical Methods for Partial Differential Equations, Vol. 75, Springer Science & Business Media, 1995.
- [20] Jakob Gawlikowski, Cedrique Rovile Njiteucheu Tassi, Mohsin Ali, Jongseok Lee, Matthias Humt, Jianxiang Feng, Anna Kruspe, Rudolph Triebel, Peter Jung, Ribana Roscher, et al., A survey of uncertainty in deep neural networks, *Artif. Intell. Rev.* 56 (Suppl 1) (2023) 1513–1589.
- [21] Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, Francesco Piccialli, Scientific machine learning through physics-informed neural networks: Where we are and what's next, *J. Sci. Comput.* 92 (3) (2022) 88.
- [22] Yao Zhang, Woong Je Sung, Dimitri N. Mavris, Application of convolutional neural network to predict airfoil lift coefficient, in: 2018 AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference, 2018, p. 1903.
- [23] Azadeh Gholami, Hossein Bonakdari, Amir Hossein Zaji, Ali Akbar Akhtari, A comparison of artificial intelligence-based classification techniques in predicting flow variables in sharp curved channels, *Eng. Comput.* 36 (2020) 295–324.
- [24] Maziar Raissi, Paris Perdikaris, George E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [25] Pao-Hsiung Chiu, Jian Cheng Wong, Chinchun Ooi, My Ha Dao, Yew-Soon Ong, CAN-PINN: A fast physics-informed neural network based on coupled-automatic-numerical differentiation method, *Comput. Methods Appl. Mech. Engrg.* 395 (2022) 114909.
- [26] Junjun Yan, Xinhai Chen, Zhichao Wang, Enqiang Zhou, Jie Liu, Auxiliary-tasks learning for physics-informed neural network-based partial differential equations solving, 2023, arXiv preprint [arXiv:2307.06167](https://arxiv.org/abs/2307.06167).
- [27] Zhichao Wang, Xinhai Chen, Tiejun Li, Chunye Gong, Yufei Pang, Jie Liu, Evaluating mesh quality with graph neural networks, *Eng. Comput.* 38 (5) (2022) 4663–4673.

- [28] Haoxuan Zhang, Haisheng Li, Nan Li, Xiaochuan Wang, MQNet: A mesh quality evaluation neural network based on dynamic graph attention, 2023, arXiv preprint [arXiv:2309.01067](https://arxiv.org/abs/2309.01067).
- [29] A. Egemen Yilmaz, Mustafa Kuzuoglu, A particle swarm optimization approach for hexahedral mesh smoothing, *Internat. J. Numer. Methods Fluids* 60 (1) (2009) 55–78.
- [30] Abel Gargallo-Peiró, Xevi Roca, Josep Sarrate, A surface mesh smoothing and untangling method independent of the CAD parameterization, *Comput. Mech.* 53 (2014) 587–609.
- [31] D.A. Lowther, D.N. Dyck, A density driven mesh generator guided by a neural network, *IEEE Trans. Magn.* 29 (2) (1993) 1927–1930.
- [32] Zheyang Zhang, Yongxing Wang, Peter K. Jimack, He Wang, MeshingNet: A new mesh generation method based on deep learning, in: *International Conference on Computational Science*, Springer, 2020, pp. 186–198.
- [33] Keefe Huang, Moritz Krügener, Alistair Brown, Friedrich Menhorn, Hans-Joachim Bungartz, Dirk Hartmann, Machine learning-based optimal mesh generation in computational fluid dynamics, 2021, arXiv preprint [arXiv:2102.12923](https://arxiv.org/abs/2102.12923).
- [34] Alexis Papagiannopoulos, Pascal Clausen, Francois Avellan, How to teach neural networks to mesh: Application on 2-D simplicial contours, *Neural Netw.* 136 (2021) 152–179.
- [35] Lu Peng, WANG Nianhua, Xinghua Chang, Laiping Zhang, WU Yadong, An automatic isotropic/anisotropic hybrid grid generation technique for viscous flow simulations based on an artificial neural network, *Chin. J. Aeronaut.* 35 (4) (2022) 102–117.
- [36] Minseong Kim, Jaeseung Lee, Jibum Kim, GMR-Net: GCN-based mesh refinement framework for elliptic pde problems, *Eng. Comput.* (2023) 1–17.
- [37] Xinhai Chen, Jie Liu, Qingyang Zhang, Jianpeng Liu, Qinglin Wang, Liang Deng, Yufei Pang, Developing a novel structured mesh generation method based on deep neural networks, *Phys. Fluids* 35 (9) (2023).
- [38] William J. Gordon, Charles A. Hall, Construction of curvilinear co-ordinate systems and applications to mesh generation, *Internat. J. Numer. Methods Engrg.* 7 (4) (1973) 461–477.
- [39] Joe F. Thompson, Zahir U.A. Warsi, C. Wayne Mastin, *Numerical Grid Generation: Foundations and Applications*, Elsevier North-Holland, Inc., 1985.
- [40] Joe F. Thompson, Bharat K. Soni, Nigel P. Weatherill, *Handbook of Grid Generation*, CRC Press, 1998.
- [41] Alex Kendall, Yarin Gal, Roberto Cipolla, Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7482–7491.
- [42] Yong Wang, Yanzhong Yao, Jiawei Guo, Zhiming Gao, A practical PINN framework for multi-scale problems with multi-magnitude loss terms, *J. Comput. Phys.* 510 (2024) 113112.